21CSC558J DEEP LEARNING APPROACHES

Module-1 - Fundamentals of Deep Learning

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Module-1 - Fundamentals of Deep Learning

- Introduction to neural networks
- Machine learning vs deep learning
- Biological motivation of neural networks
- Fundamentals of tensor flow-data structures in tensor flow,
- Perceptron model-perceptron learning rule,
- Learning Ex-OR problem,
- Activation functions- role of activation function, different types of activation functions,
- Feedforward neural networks, back propagation in neural networks-chain rule, loss functions- binary cross entropy loss, categorical cross entropy loss
- Lab 1: exploring the environment of tensor flow
- Lab 2: Building Programs to Perform Basic Operations in Tensors, regression

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

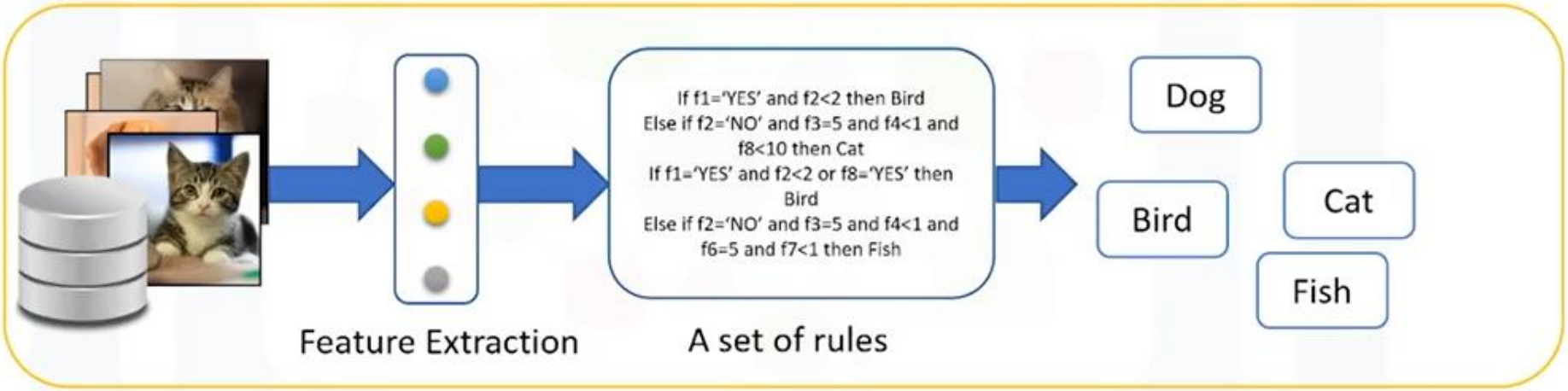
What is Machine Learning?

- Machine learning (ML) is a subset of artificial intelligence (AI) that enables computers to learn from data without being explicitly programmed.
- Instead of following a fixed set of instructions, machine learning models adjust based on the input they receive and make predictions or decisions based on this learning process.

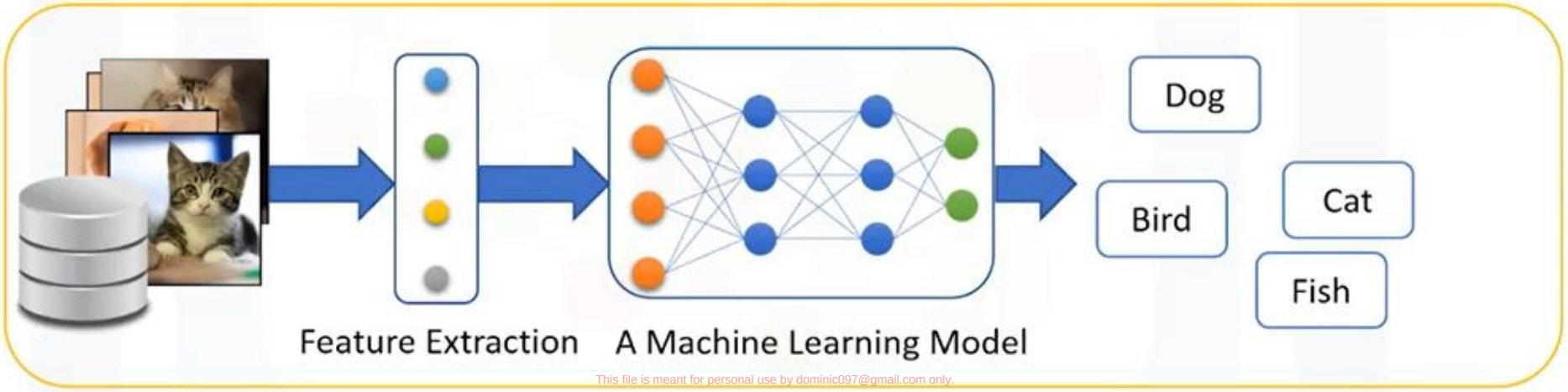
This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

How Machine Learning Works?

Traditional Model

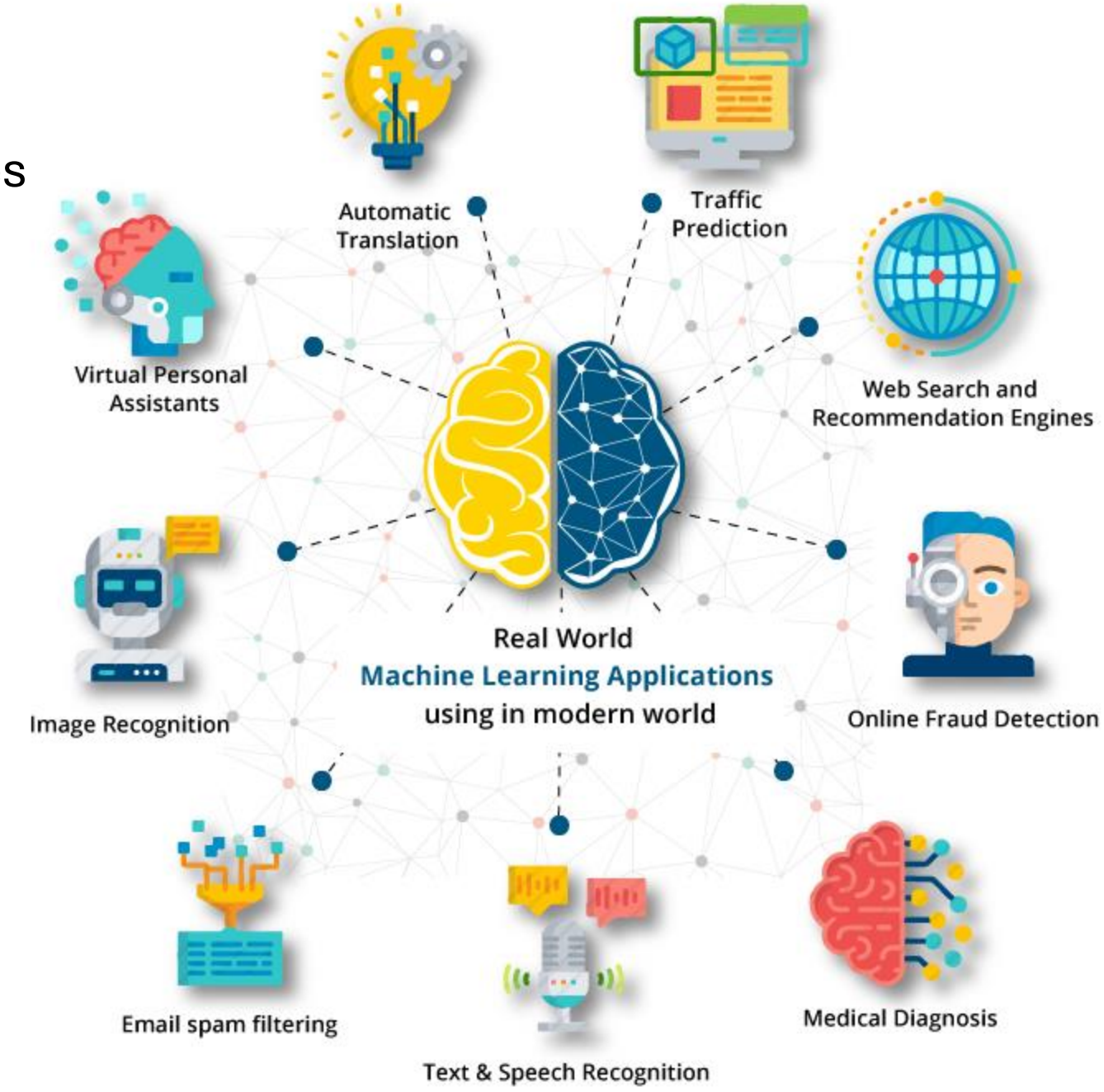


Machine Learning Model



This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Machine Learning Examples



This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Machine Learning Techniques

- Regression / Estimation
  - Predicting continuous values
- Classification
  - Predicting item class / category of the case
- Clustering
  - Finding the structure of the data
- Associations
  - Associating frequent co-occuring items / events

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Machine Learning Techniques

- Anomaly detection
  - Discovering abnormal and unusual cases
- Sequence mining
  - Predicting next events
- Dimension reduction
  - Reducing the size of data
- Recommendation systems
  - Recommending items

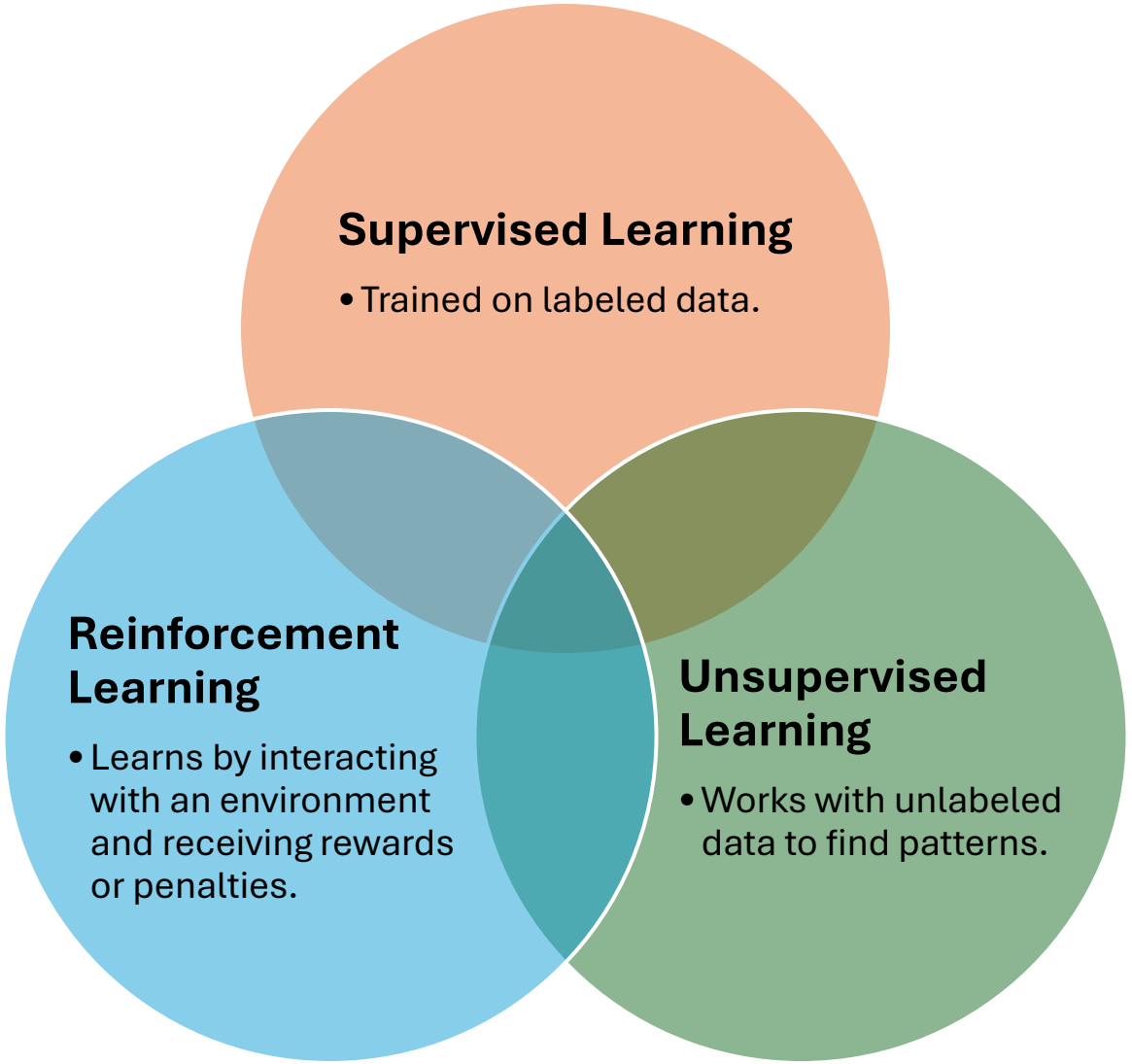
This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Difference between AI, ML, and DL

AI	ML	DL
• AI encompasses **all intelligent systems**, including rule-based systems (which don't learn from data) and machine learning systems.	• ML is a more specialized area within AI that **uses data to teach models** to perform tasks like classification, regression, clustering, etc.	• DL is a specialized subset of ML focused on **using neural networks** with many layers to automatically learn features and representations from raw data

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

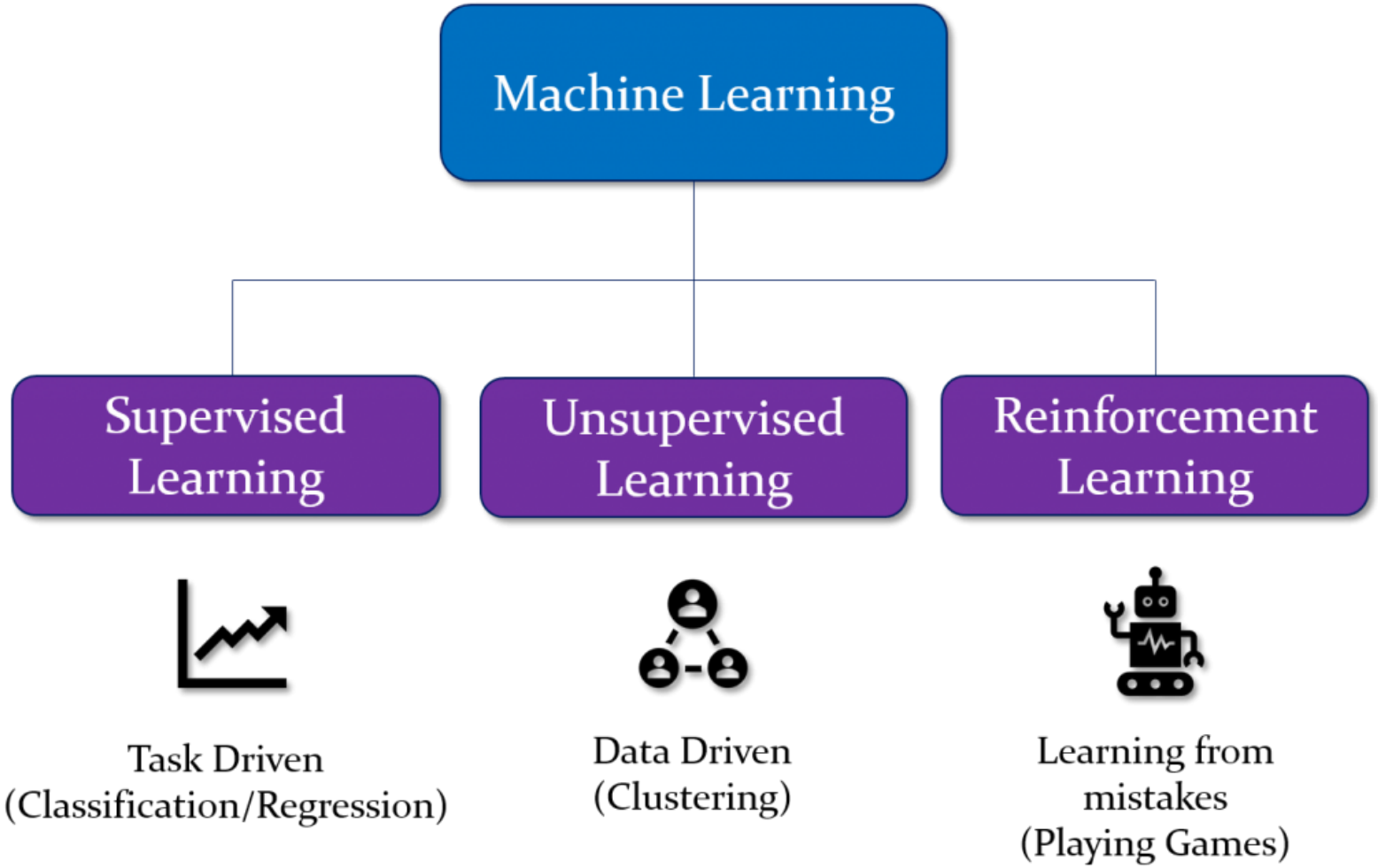
Types of Machine Learning



Dr.M.Sivakumar, SRMIST

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Types of Machine Learning



This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Types of Machine Learning

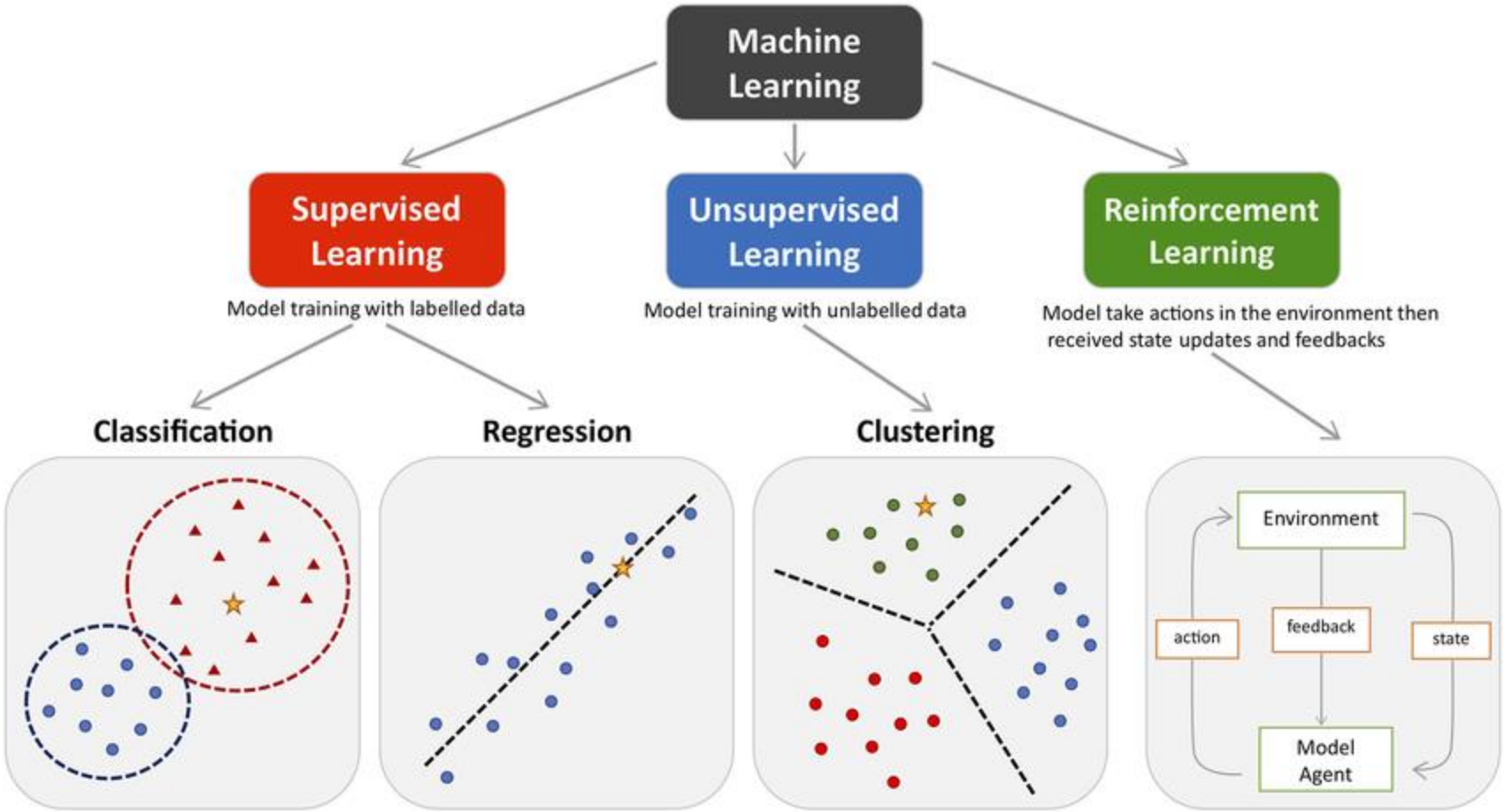
SUPERVISED LEARNING

UNSUPERVISED LEARNING

REINFORCEMENT LEARNING

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

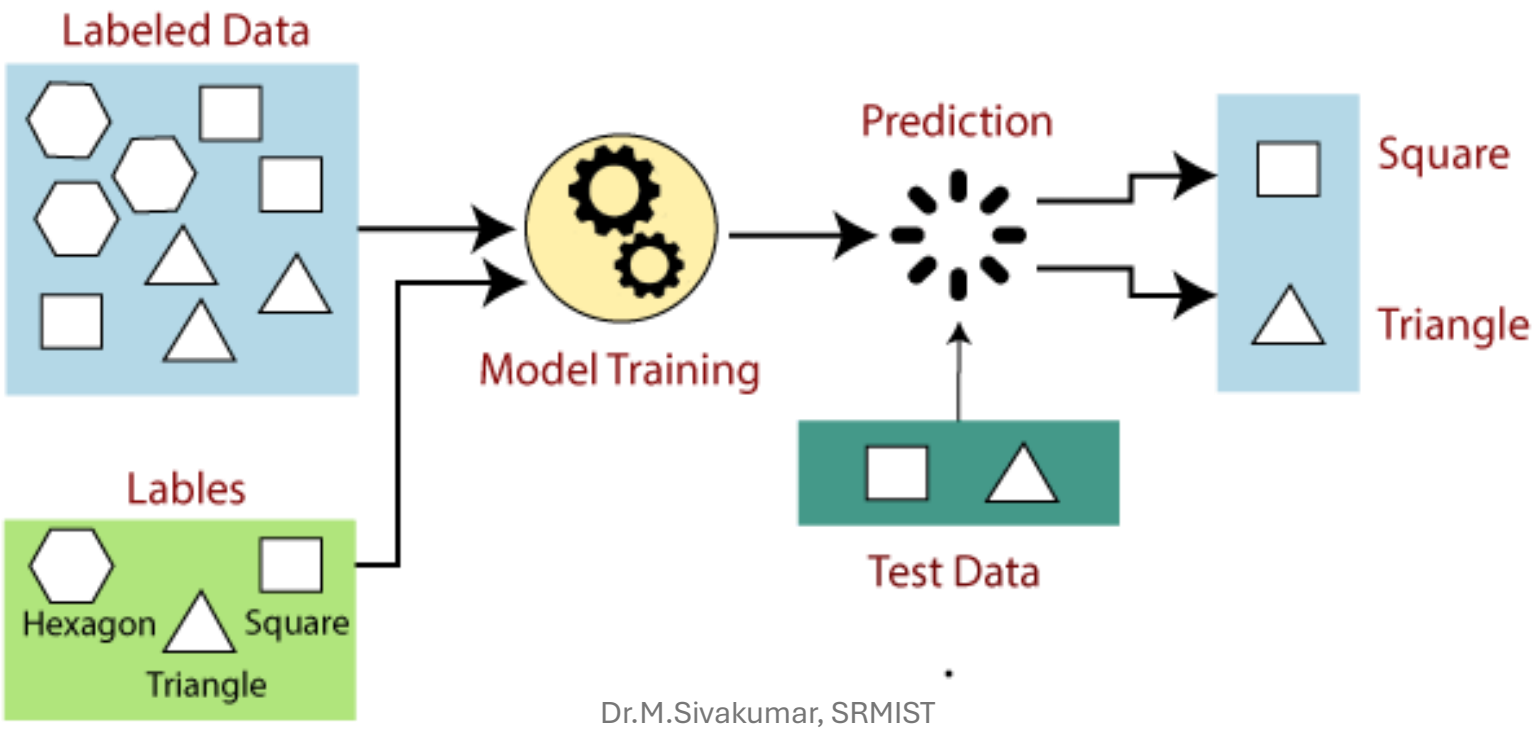
Types of Machine Learning



This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

What is supervised learning?

- We “teach the model”, then with that knowledge it can predict unknown or future instances



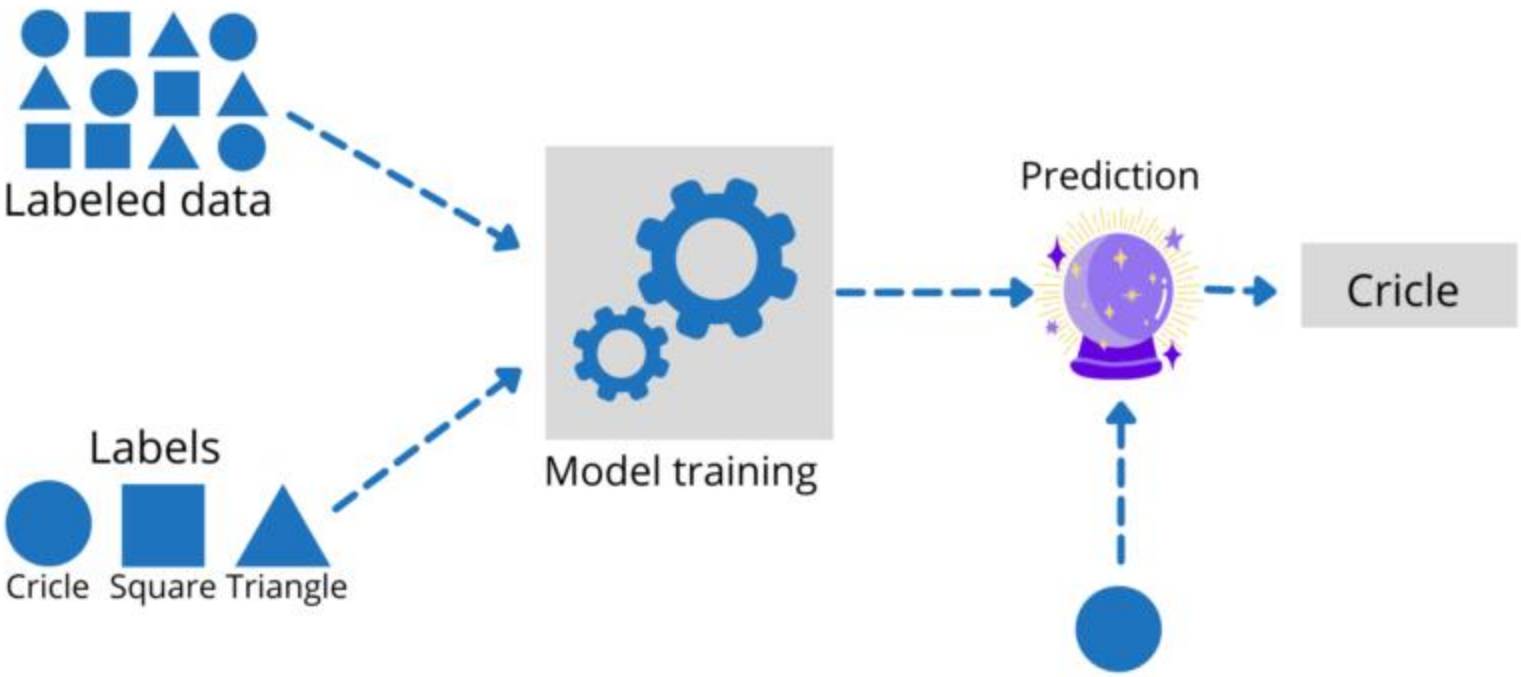
This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Teaching the model with labeled data

ID	Clump	UnifSize	UnifShape	MargAdh	SingEpiSize	BareNuc	BlandChrom	NormNucl	Mit	Class
1000025	5	1	1	1	2	1	3	1	1	benign
1002945	5	4	4	5	7	10	3	2	1	benign
1015425	3	1	1	1	2	2	3	1	1	malignant
1016277	6	8	8	1	3	4	3	7	1	benign
1017023	4	1	1	3	2	1	3	1	1	benign
1017122	8	10	10	8	7	10		7	1	malignant
1018099	1	1	1	1	2	10	3	1	1	benign
1018561	2	1	2	H	2	1	3	1	1	benign
1033078	2	1	1	1	2	1	1	1	5	benign
1033078	4	2	1	1	2	1	2	1	1	benign

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Teaching the model with labeled data



This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Types of supervised learning

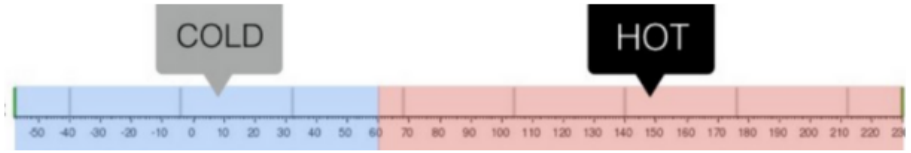
Regression

What will be the temperature tomorrow?

Fahrenheit

Classification

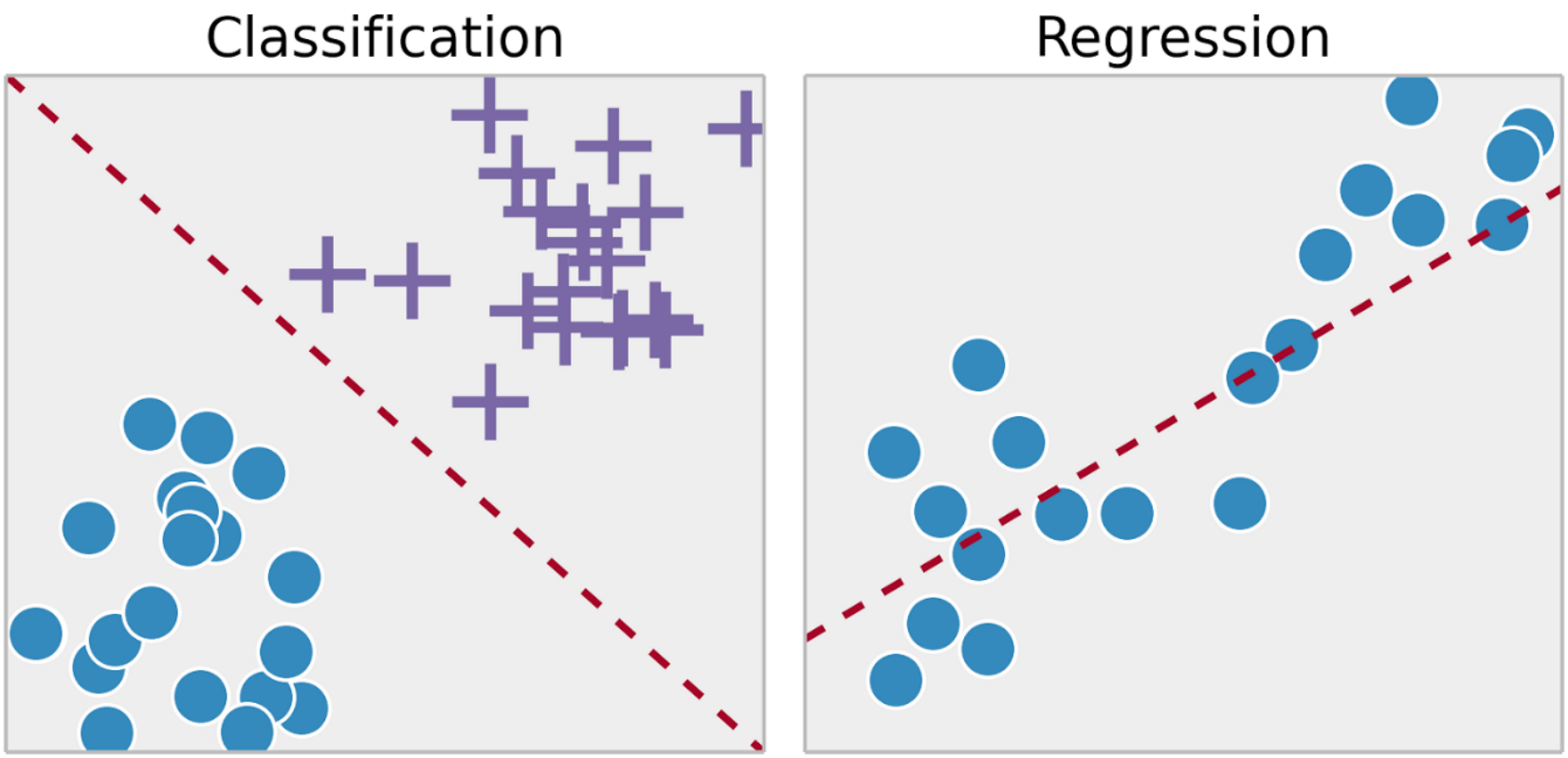
Will it be hot or cold tomorrow?



Fahrenheit

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Types of supervised learning



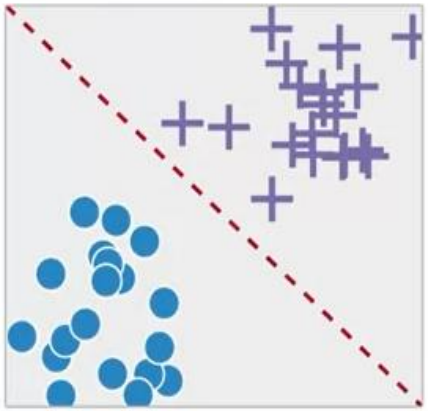
This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

What is classification?

Classification is the process of predicting discrete class labels or categories

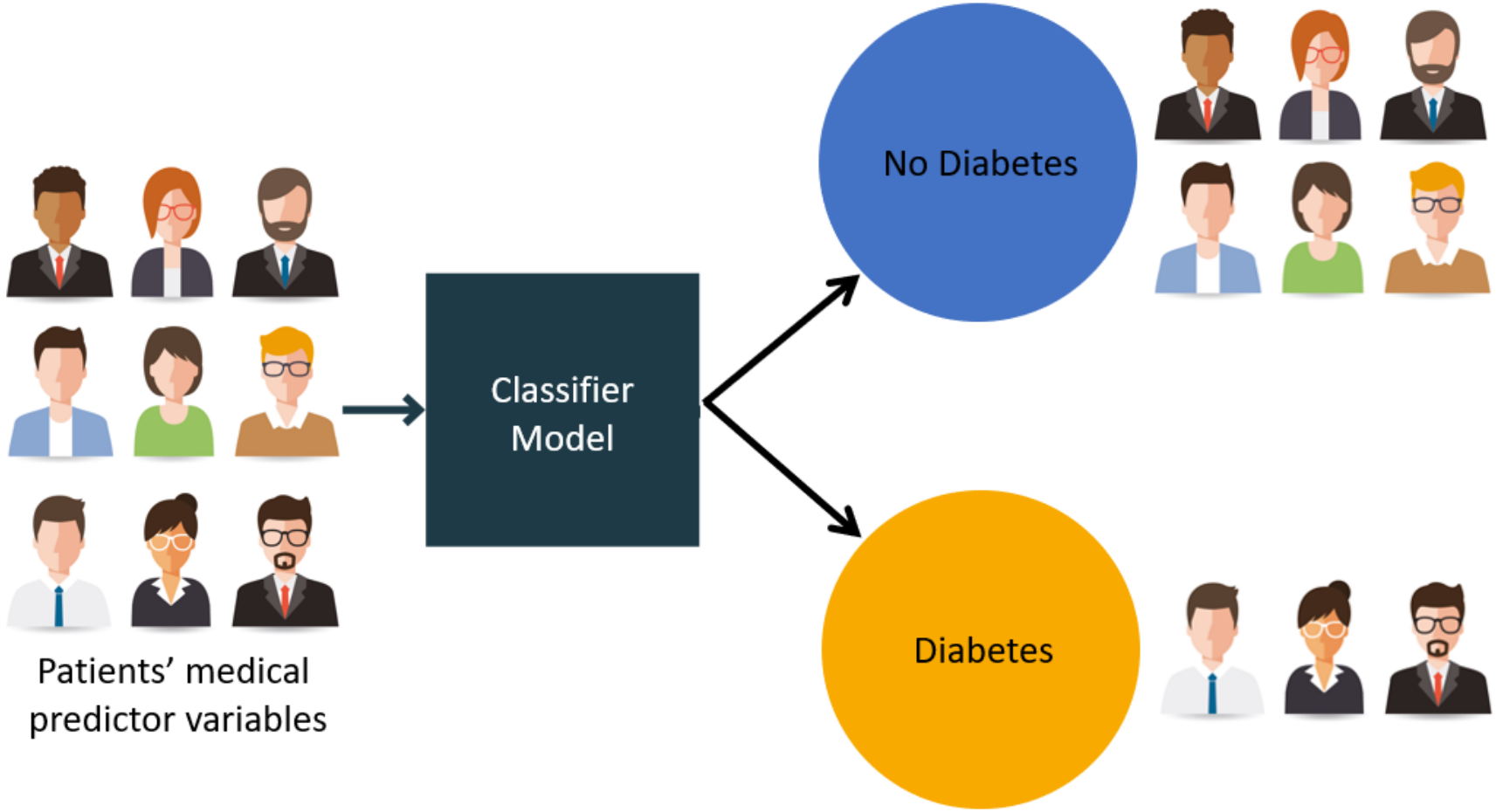
ID	Clump	UnifSize	UnifShape	MargAdh	SingEpiSize	BareNuc	BlandChrom	NormNucl	Mit	Class
1000025	5	1	1	1	2	1	3	1	1	benign
1002945	5	4	4	5	7	10	3	2	1	benign
1015425	3	1	1	1	2	2	3	1	1	malignant
1016277	6	8	8	1	3	4	3	7	1	benign
1017023	4	1	1	3	2	1	3	1	1	benign
1017122	8	10	10	8	7	10		7	1	malignant
1018099	1	1	1	1	2	10	3	1	1	benign
1018561	2	1	2	H	2	1	3	1	1	benign
1033078	2	1	1	1	2	1	1	1	5	benign
1033078	4	2	1	1	2	1	2	1	1	benign

Categorical Values



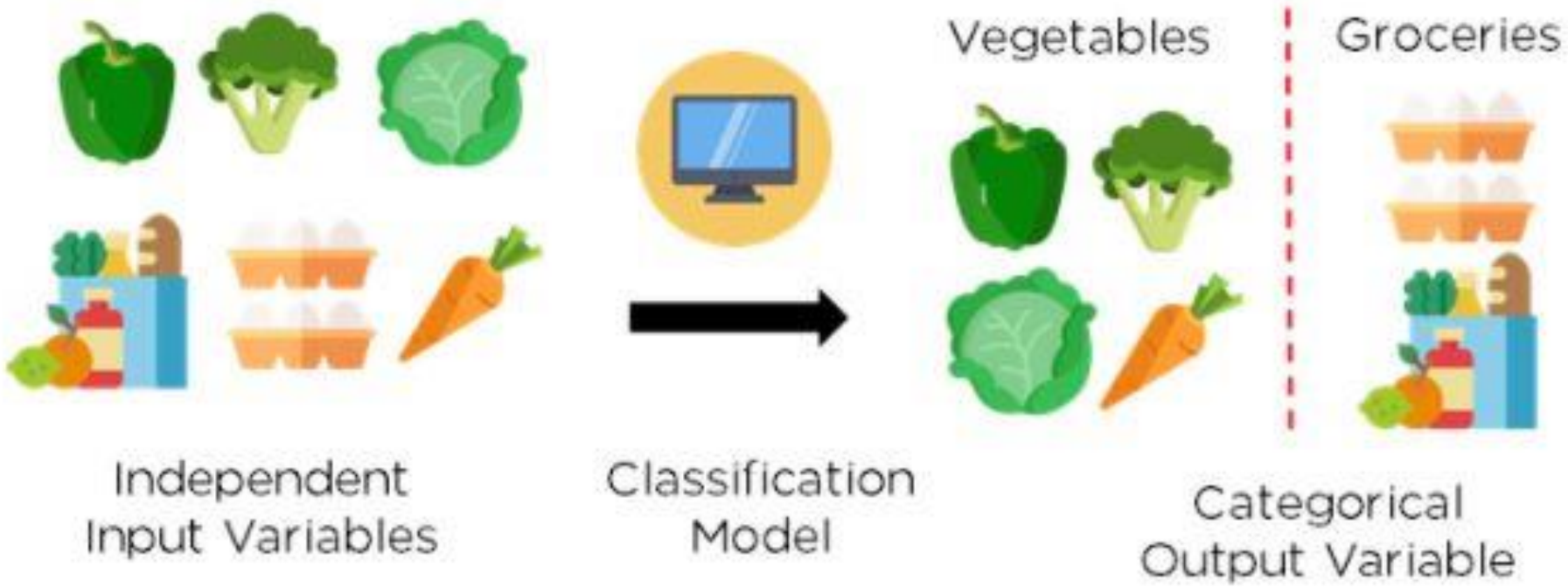
This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

What is classification?



This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

What is classification?



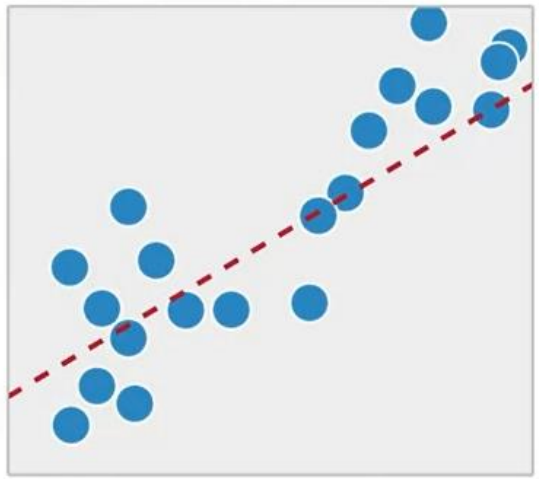
This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

What is regression?

Regression is the process of predicting continuous values

	ENGINESIZE	CYLINDERS	FUELCONSUMPTION_COMB	CO2EMISSIONS
0	2.0	4	8.5	196
1	2.4	4	9.6	221
2	1.5	4	5.9	136
3	3.5	6	11.1	255
4	3.5	6	10.6	244
5	3.5	6	10.0	230
6	3.5	6	10.1	232
7	3.7	6	11.1	255
8	3.7	6	11.6	267
9	2.4	4	9.2	?

Continuous Values



This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Unsupervised learning

- The training of machine using information that is neither classified nor labeled
- Allowing the algorithm to act on that information **without guidance.**
- To group unsorted information **according to similarities, patterns and differences** without any prior training of data.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

What is unsupervised learning?

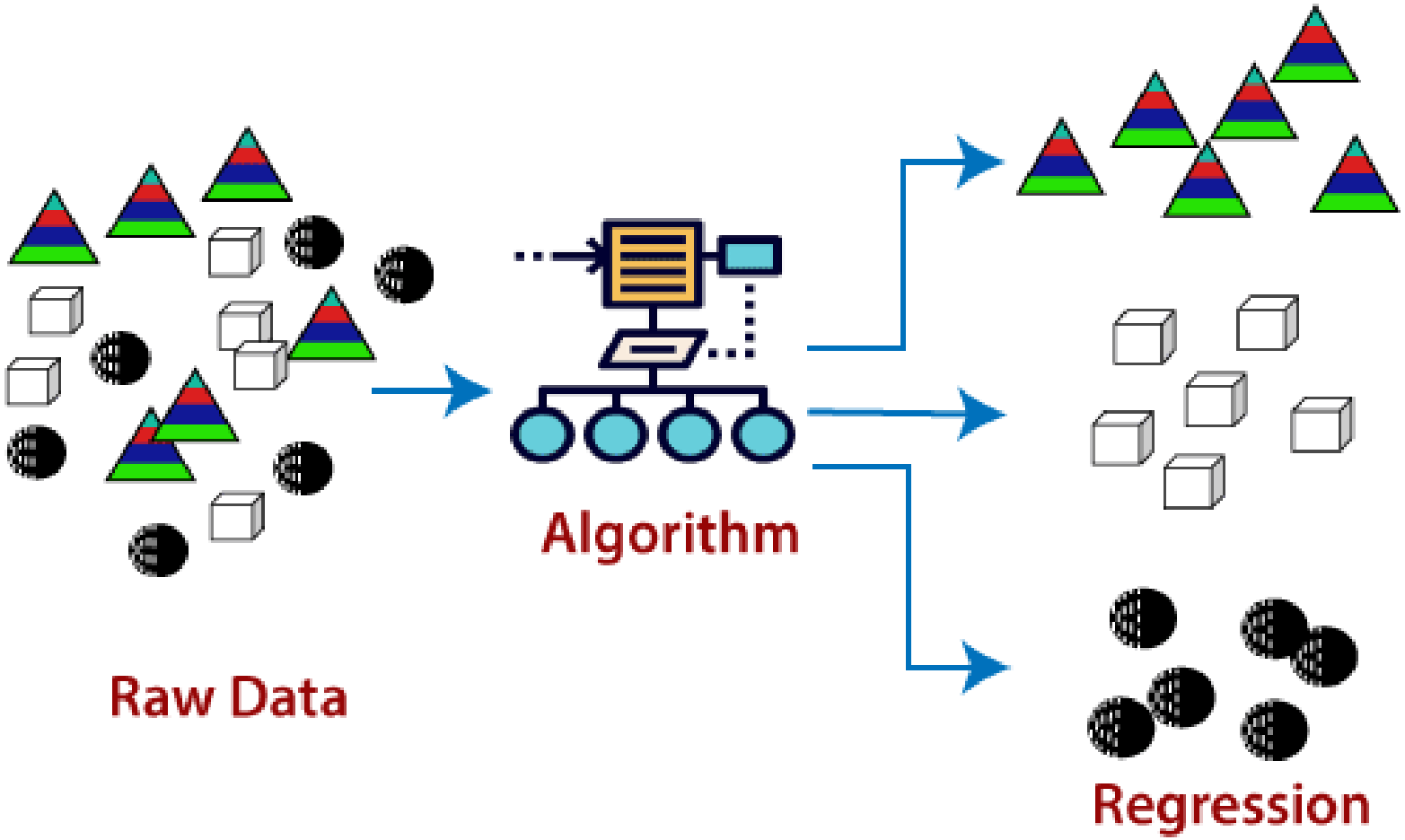
The model works on its own to discover information

Customer Id	Age	Edu	Years Employed	Income	Card Debt	Other Debt	Address	DebtIncomeRatio
1	41	2	6	19	0.124	1.073	NBA001	6.3
2	47	1	26	100	4.582	8.218	NBA021	12.8
3	33	2	10	57	6.111	5.802	NBA013	20.9
4	29	2	4	19	0.681	0.516	NBA009	6.3
5	47	1	31	253	9.308	8.908	NBA008	7.2
6	40	1	23	81	0.998	7.831	NBA016	10.9
7	38	2	4	56	0.442	0.454	NBA013	1.6
8	42	3	0	64	0.279	3.945	NBA009	6.6
9	26	1	5	18	0.575	2.215	NBA006	15.5
10	47	3	23	115	0.653	3.947	NBA011	4
11	44	3	8	88	0.285	5.083	NBA010	6.1
12	34	2	9	40	0.374	0.266	NBA003	1.6

- Unsupervised leaning techniques
  - Clustering
  - Market basket analysis
  - Dimension reduction
  - Density estimation

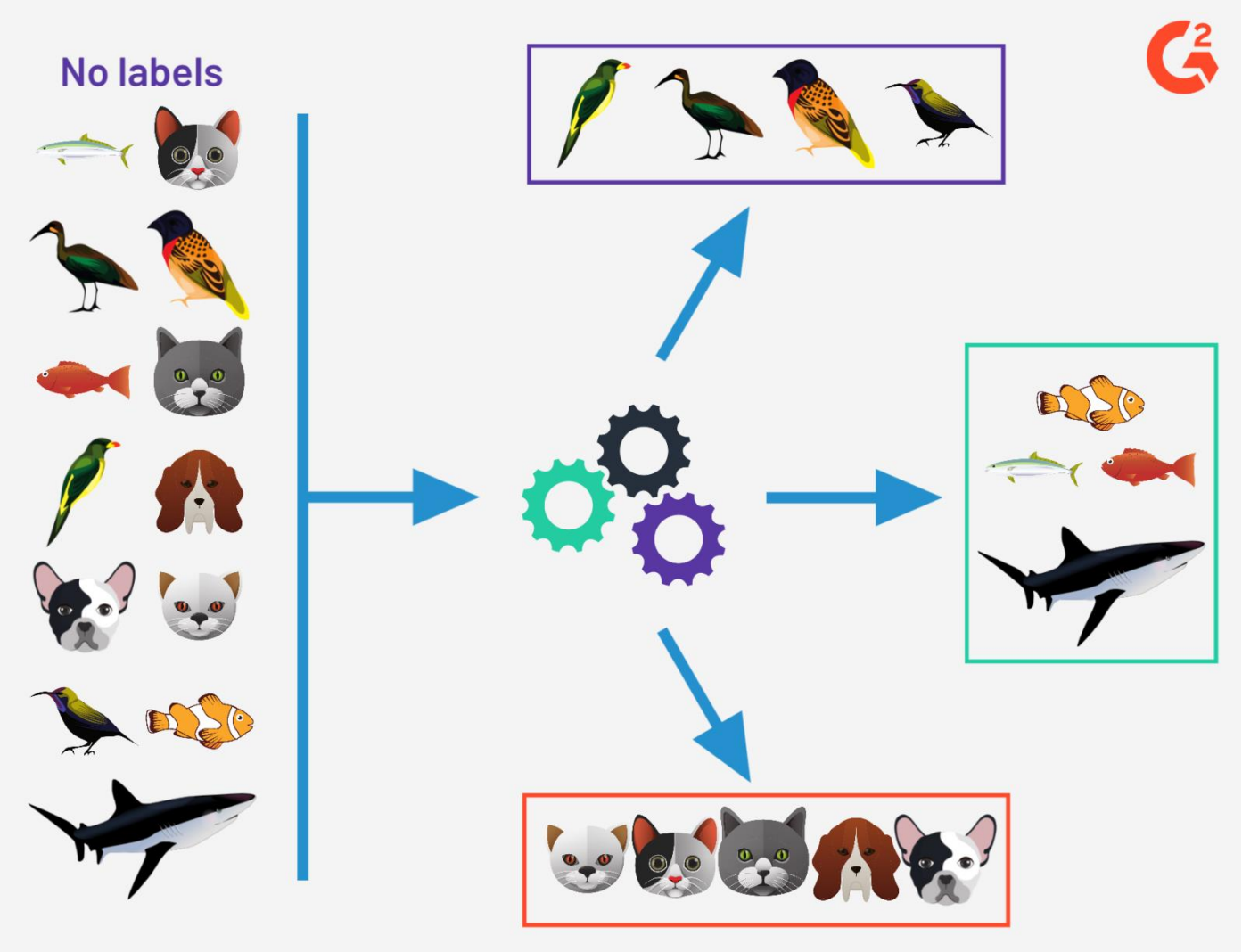
This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

What is unsupervised learning?



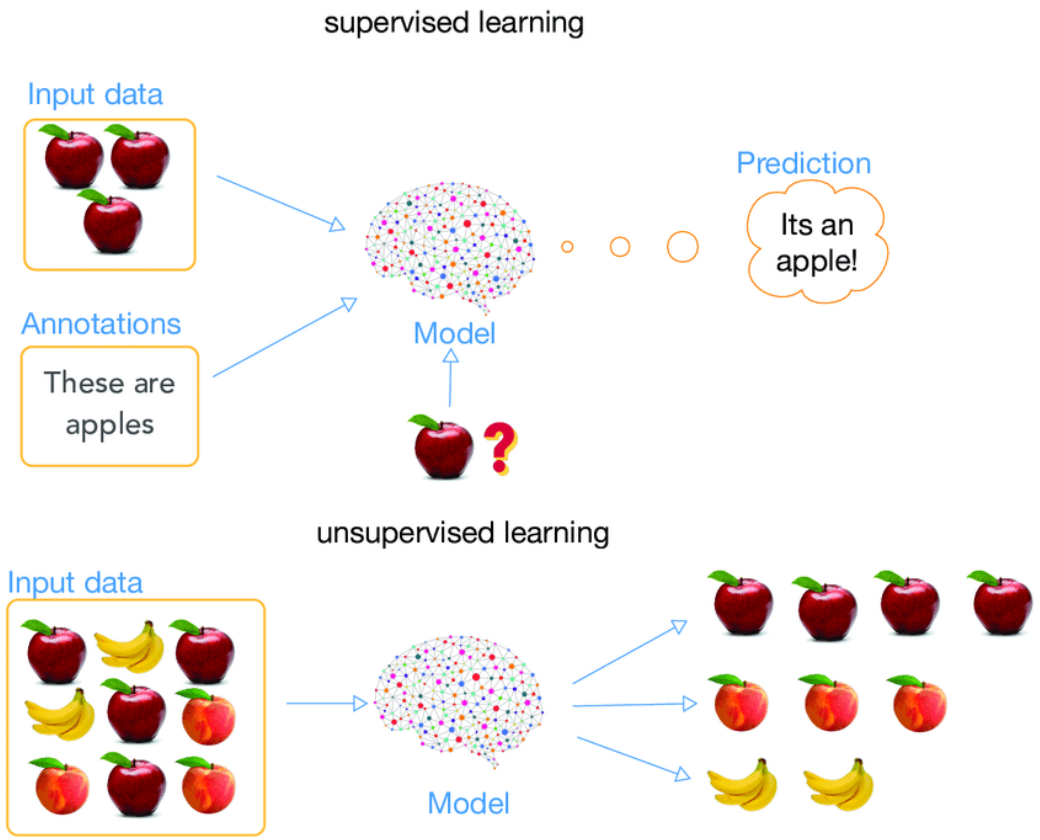
This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

What is unsupervised learning?



This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Supervised vs Unsupervised



Supervised Learning

- **Classification:** Classifies labeled data
- **Regression:** Predicts trends using previous labeled data
- Has more evaluation methods than unsupervised learning
- Controlled environment

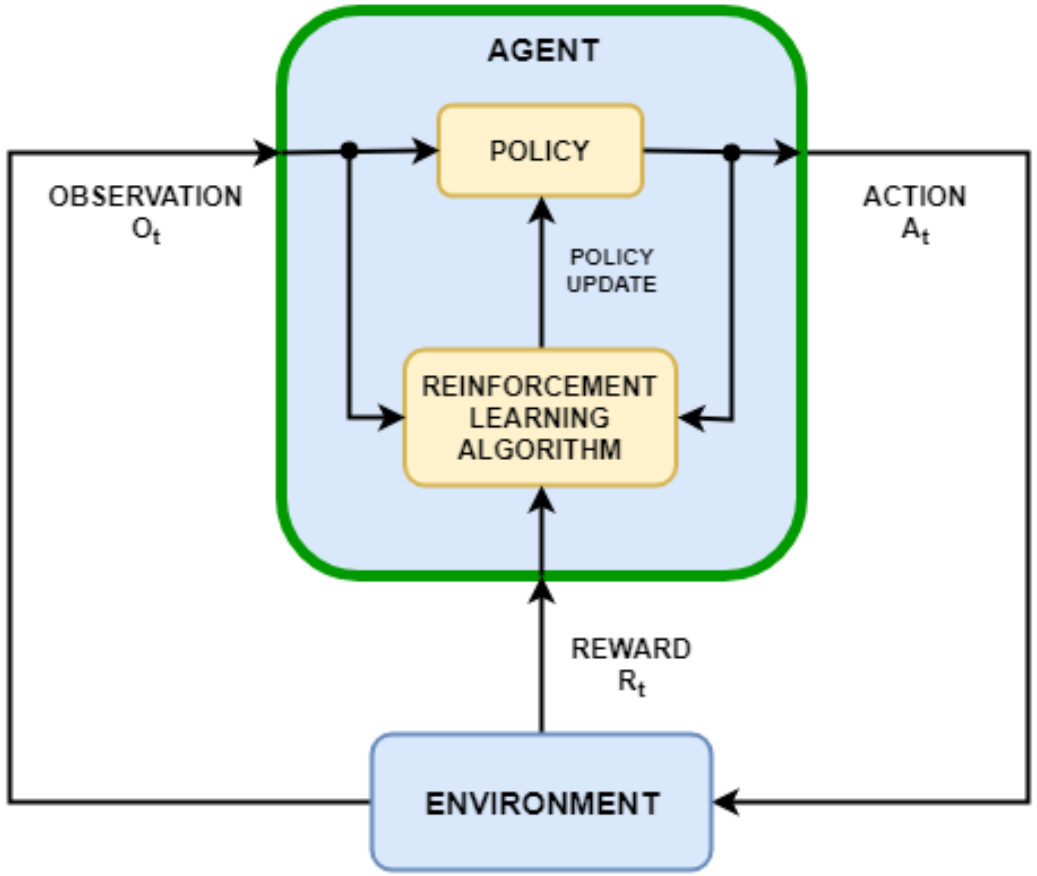
Unsupervised Learning

- **Clustering:** Finds patterns and groupings from unlabeled data
- Has fewer evaluation methods than supervised learning
- Less controlled environment

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

What is reinforcement Learning?

- An agent learn how to behave in a environment by performing actions and seeing the results
- An agent will learn from the environment by interacting with it and receiving rewards for performing actions.



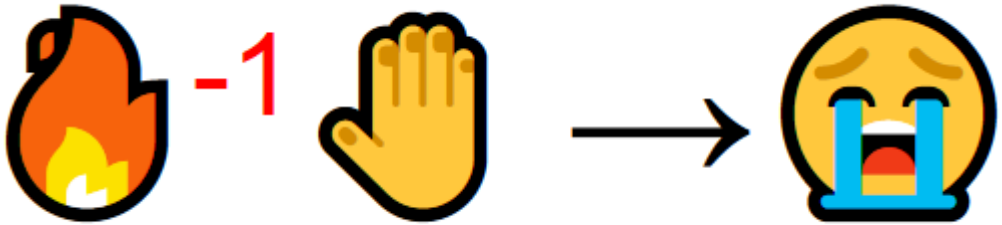
This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Example

- Imagine you're a child in a living room. You see a fireplace, and you approach it.

- It's warm, it's positive, you feel good *(Positive Reward +1).* You understand that fire is a positive thing.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.



- you try to touch the fire. Ouch! It burns your hand *(Negative reward -1)*.

- You've just understood that fire is positive when you are a sufficient distance away, because it produces warmth.
- But get too close to it and you will be burned.

- "That's how humans learn, through interaction"

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Model Evaluation

- Train and test on the same dataset
- Train/Test split

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

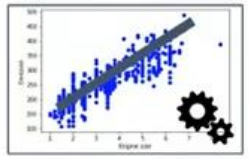
Train and test on the same dataset

	ENGINESIZE	CYLINDERS	FUELCONSUMPTION_COMB	CO2EMISSIONS
0	2.0	4	8.5	196
1	2.4	4	9.6	221
2	1.5	4	5.9	136
3	3.5	6	11.1	255
4	3.5	6	10.6	244
5	3.5	6	10.0	230
6	3.5	6	10.1	232
7	3.7	6	11.1	255
8	3.7	6	11.6	267
9	2.4	4	9.2	212

Train

Test

Actual values



	Prediction
6	234
7	256
8	267
9	210

Predicted values

Activate Windows
Go to Settings to activate Windows.

$$Error = \frac{(232 - 234) + (255 - 256) + \ \ldots}{4}$$

$$Error = \frac{1}{n}\sum_{j=1}^{n}|y_j - \hat{y}_j|$$

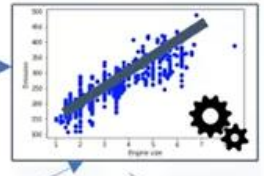
This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Train/Test split

	ENGINESIZE	CYLINDERS	FUELCONSUMPTION_COMB	CO2EMISSIONS
0	2.0	4	8.5	196
1	2.4	4	9.6	221
2	1.5	4	5.9	136
3	3.5	6	11.1	255
4	3.5	6	10.6	244
5	3.5	6	10.0	230
6	3.5	6	10.1	232
7	3.7	6	11.1	255
8	3.7	6	11.6	267
9	2.4	4	9.2	212

Train

Test



	Prediction
6	234
7	256
8	267
9	210

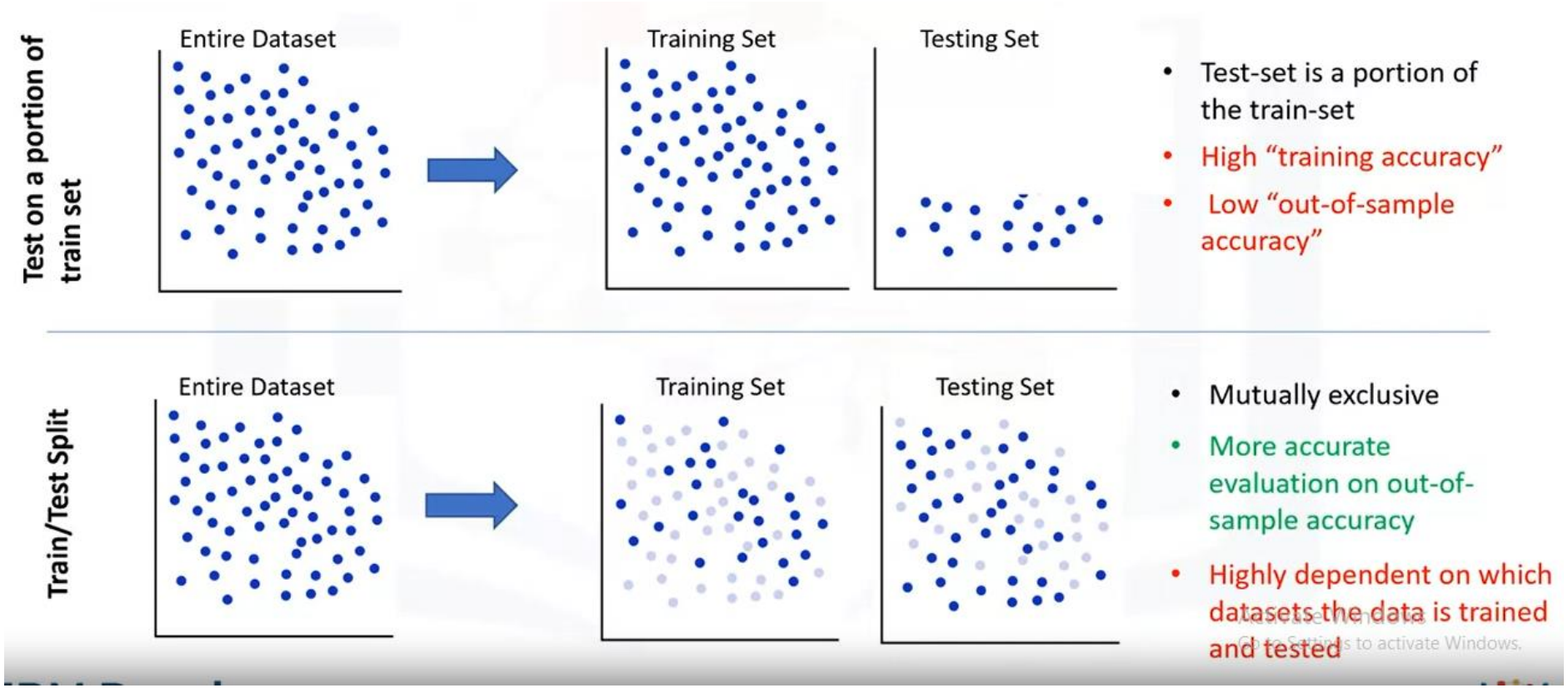
Actual values

Predicted values

Activate Windows
Go to Settings to activate Windows.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Which is best?



- Test-set is a portion of the train-set
- High "training accuracy"
- Low "out-of-sample accuracy"

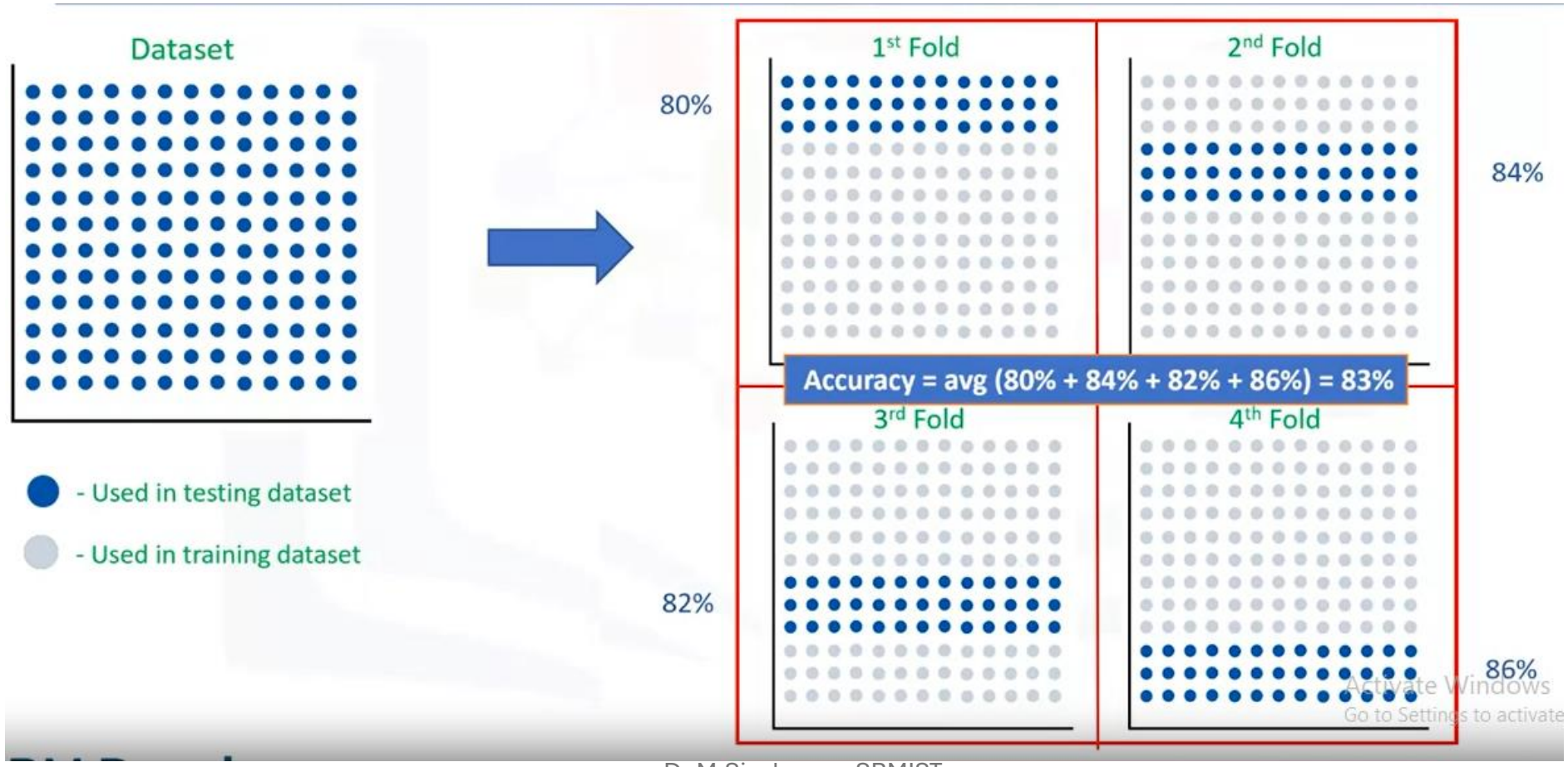
- Mutually exclusive
- More accurate evaluation on out-of-sample accuracy
- Highly dependent on which datasets the data is trained and tested

Activate Windows
Go to Settings to activate Windows.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

K-fold cross-validation

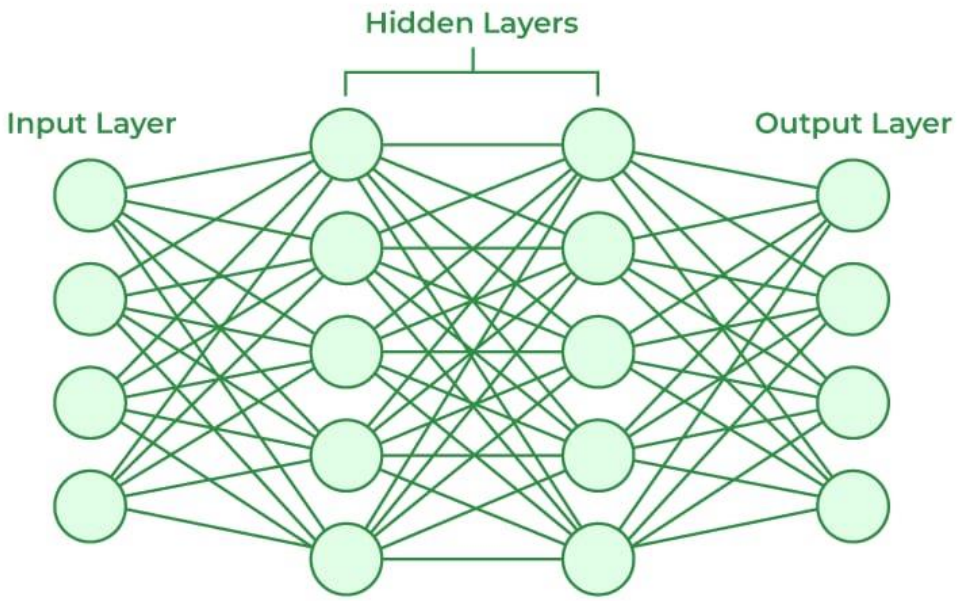
4-fold cross-validation(cv)



This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Deep Learning

- What is Deep Learning?
  - Deep Learning mimics the working of the human brain using layered neural networks to learn complex relationships in data.
  - Instead of manually extracting features, deep learning **automatically learns features** from raw data



This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Why is it Called “Deep”?

- “Deep” refers to **multiple hidden layers** between input and output.
- Traditional ML → 1–2 layers
  Deep Learning → 5, 10, 50+ layers
- More layers → Learn complex patterns.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Machine learning vs deep learning

Machine Learning	Deep Learning
Needs manual feature extraction	Learns features automatically
Works with small datasets	Needs large datasets
Less computational power	Requires GPU / High compute

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Machine learning vs deep learning

- Example 1 – Spam Detection
  - **ML Approach**
    - Extract text features
    - Train classifier
  - **DL Approach**
    - Feed raw email text
    - Use deep NLP model
- **Example 2 – Medical Diagnosis**
  - **ML**
    - Doctor + Engineer define features:
    - Tumor size
    - Shape
    - Texture
  - **DL**
    - System learns directly from:
    - MRI images
    - CT scans

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Machine learning vs deep learning

When to Use Machine Learning?

- Small dataset
- Need explainability
- Less computational power
- Structured data (tables)

When to Use Deep Learning?

- Large dataset
- Complex data (image, audio, video, NLP)
- High accuracy needed
- GPU available

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Components of Deep Learning

- Artificial Neural Networks (ANN)
- Activation Functions
- Loss Function
- Optimizer

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

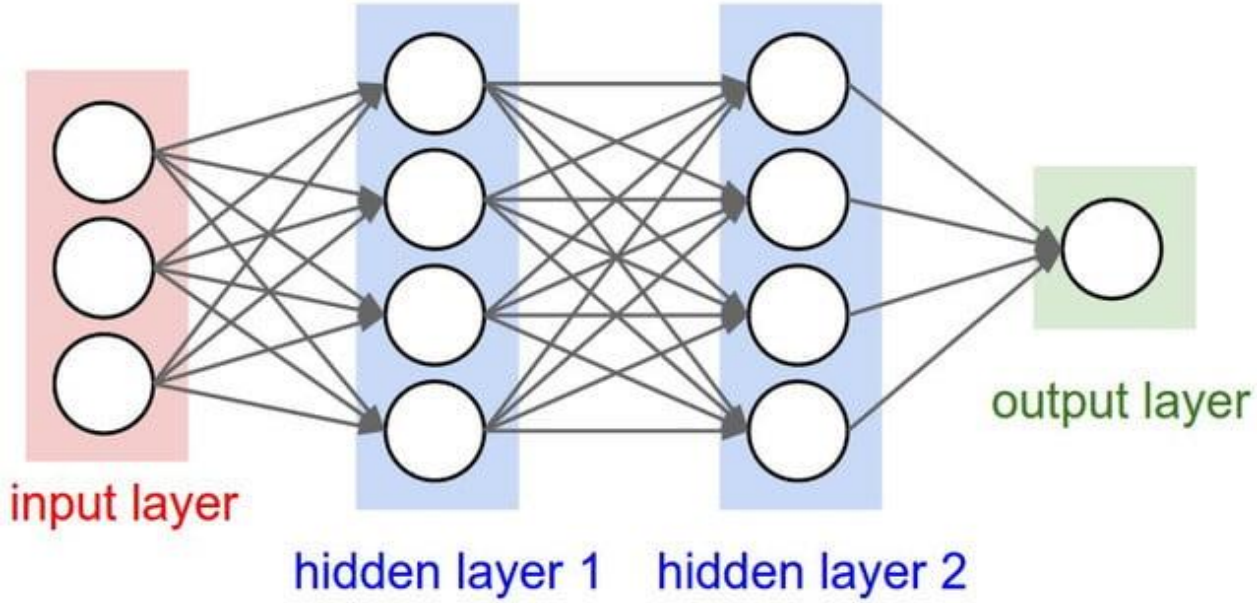
Deep Learning Architectures

- Convolutional Neural Network (CNN)
  - Used for Image Recognition, Medical Imaging, Face Detection
- Recurrent Neural Network (RNN)
  - Used for Speech Recognition, Time Series Prediction, NLP Tasks
- Transformer Models
  - Used for Chatbots, Language Translation, Large Language Models

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Artificial Neural Networks (ANN)

- Artificial Neural networks are computer models inspired by the human brain
- Made up of connected processing units called neurons
- Learn patterns from data automatically
- Used in image recognition, speech recognition, healthcare, finance



This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

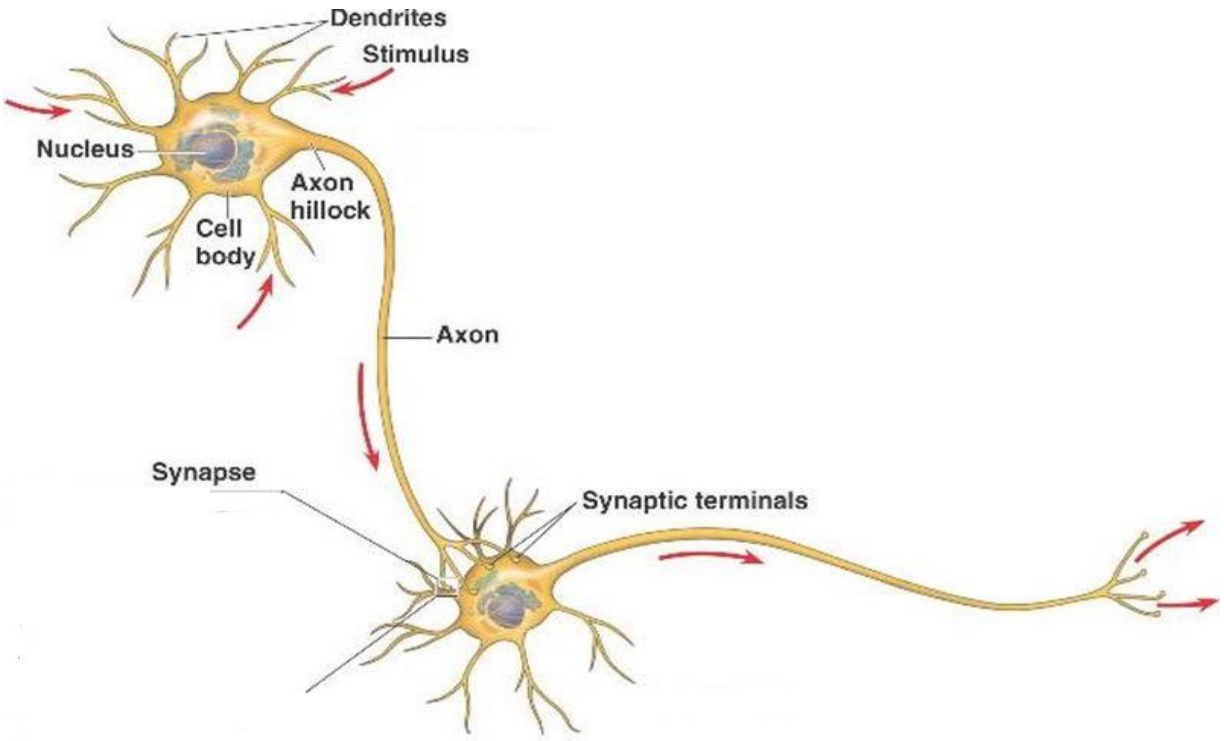
What is ANN?

- Artificial Neural Networks are the **foundation architecture** of deep learning models.
- They consist of:
  - Input Layer → Receives raw data
  - Hidden Layers → Perform feature extraction
  - Output Layer → Produces prediction
- Each neuron performs:
  - Output = Activation(Σ(weight × input) + bias)

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

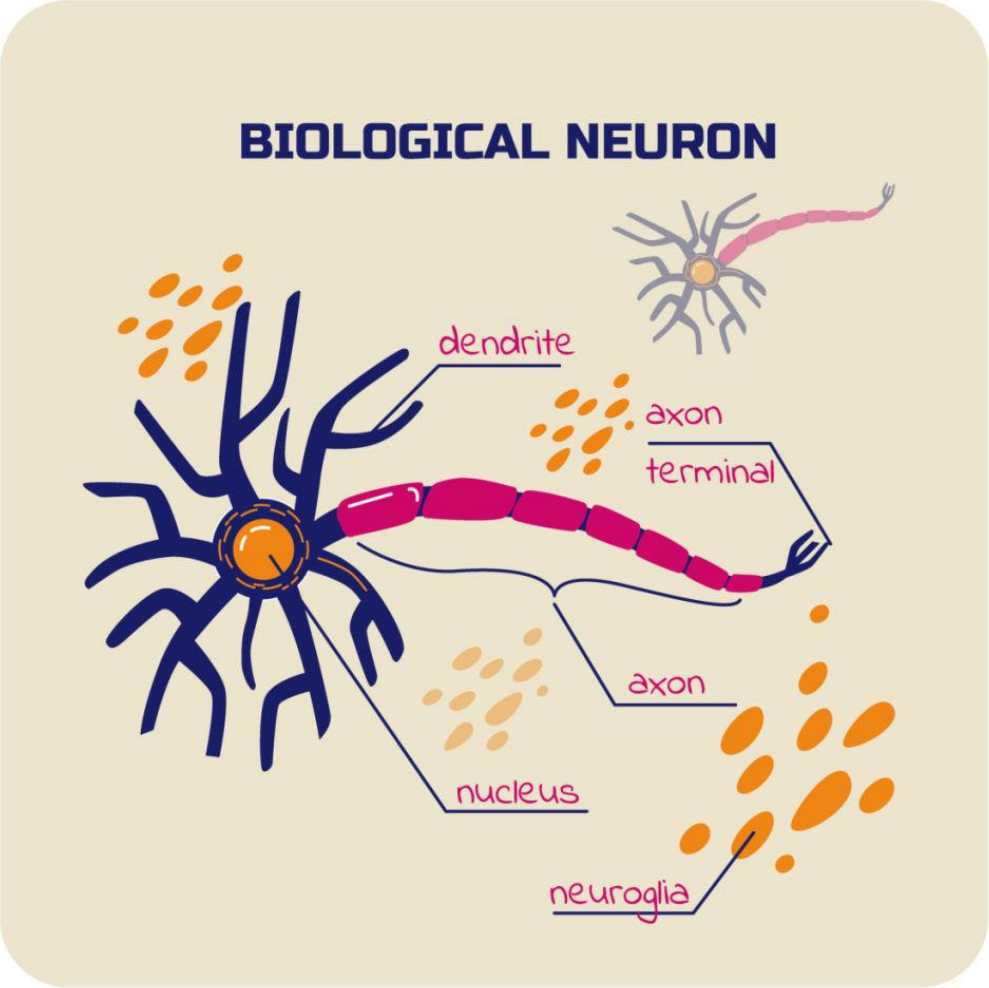
Biological motivation of neural networks

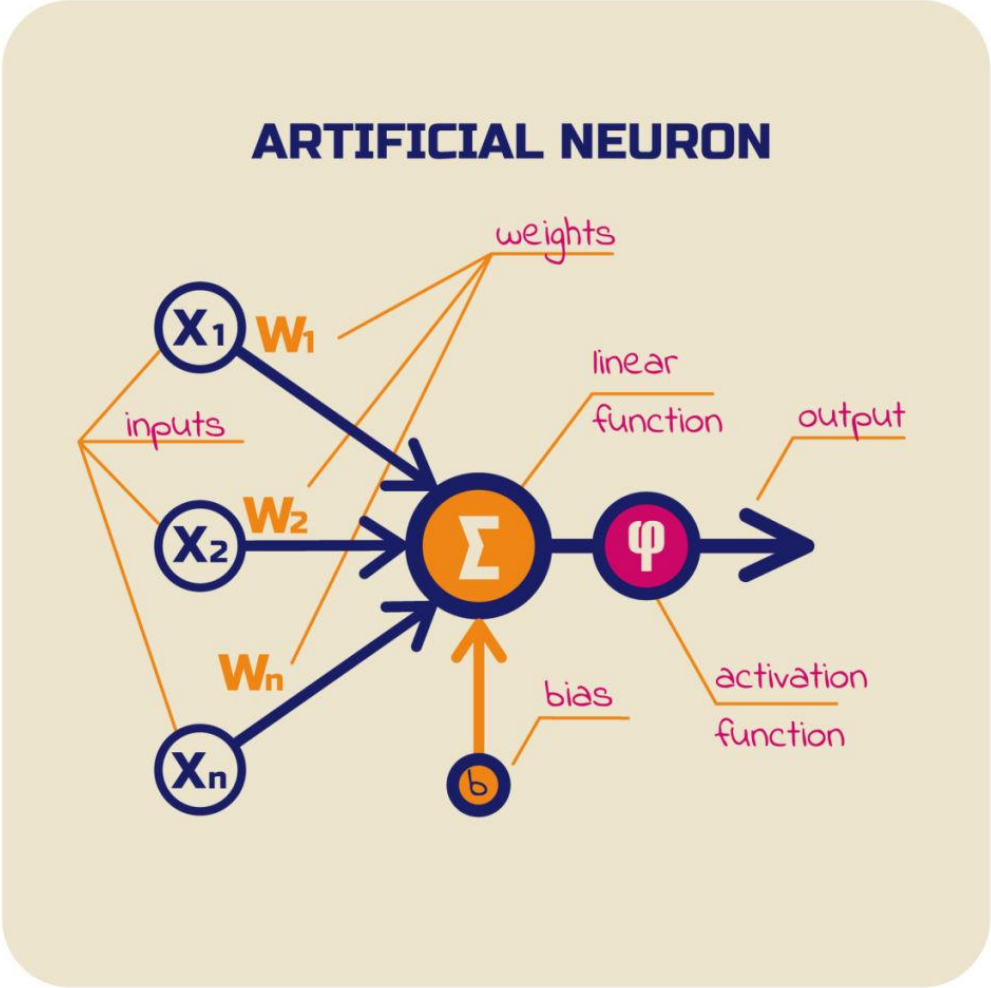
- Inspired by human brain neurons
- Learning happens by changing weights
- **Dendrites**
  - responsible for receiving the information from other neurons it is connected to
  - They are like the ears of neuron.
- **Soma**
  - It is the cell body of the neuron
  - responsible for processing of information, they have received from dendrites.
- **Axon –**
  - It is just like a cable through which neurons send the information.
- **Synapses**
  - It is the connection between the axon and other neuron dendrites.



This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

BNN Vs ANN





This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

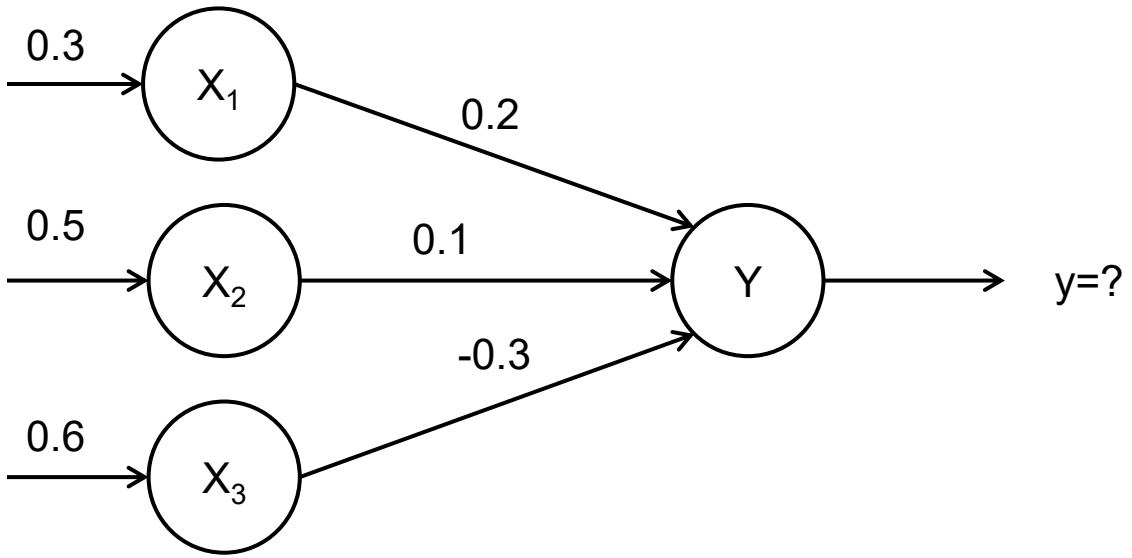
BNN Vs ANN

Biological Neural Network (BNN)	Artificial Neural Network (ANN)
Soma	Node
Dendrites	Input
Synapse	Weights or Interconnections
Axon	Output

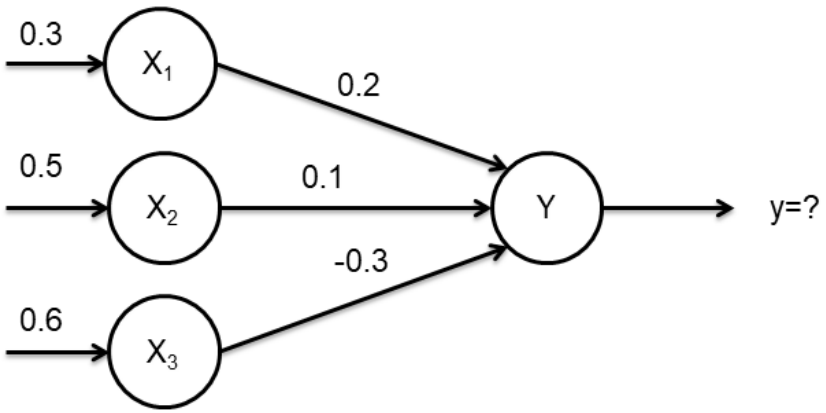
This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Problem-1

- Calculate net input to the output neuron for the network shown below.



This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.



Solution:

Given data,

[x1,x2,x3]=[0.3,0.5,0.6]

[w1,w2,w3]=[0.2,0.1,-0.3]

The net input can be calculated as

$$y_{in} = x_1w_1+x_2w_2+x_3w_3$$

$$= 0.3 \times 0.2 + 0.5 \times 0.1 + 0.6 \times (-0.3)$$

$$= 0.06 + 0.05 – 0.18$$

$$= -0.07$$

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Fundamentals of Tensor Flow

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

TensorFlow

TensorFlow is one of the most widely used frameworks for Machine Learning and Deep Learning development.

It provides tools to build, train, optimize, and deploy AI models efficiently.

TensorFlow is developed by Google and supports large-scale numerical computation using tensors and computational graphs.

The Google Brain team initially developed the TensorFlow Python deep-learning library for internal use.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

What is TensorFlow?

TensorFlow is an **open-source numerical computation and machine learning framework** that uses data flow graphs and tensors to perform operations.

TensorFlow boasts of a flexible and comprehensive collection of libraries, tools, and community resources.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

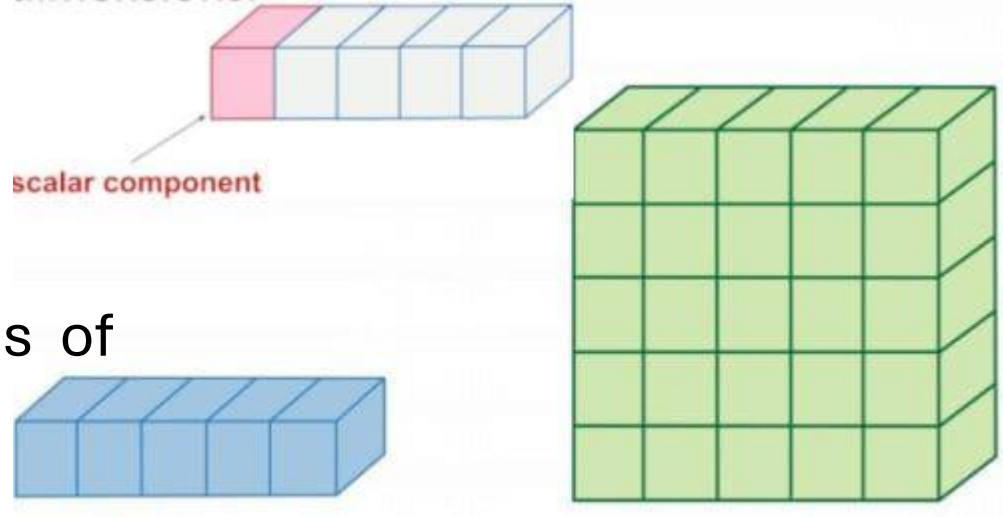
TensorFlow Basics

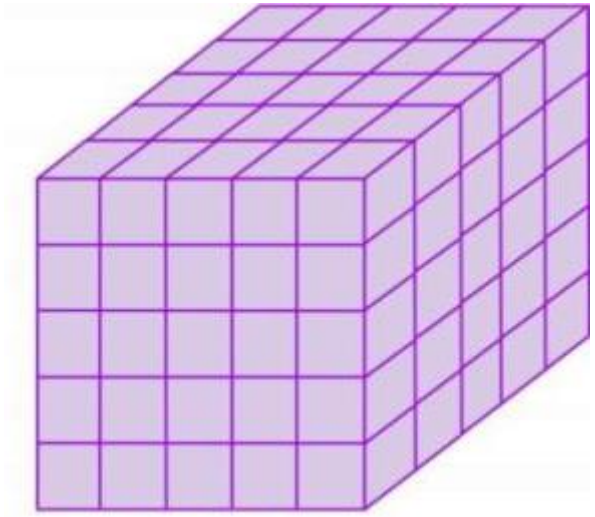
- Tensors
- Shape
- Type
- Graph
- Session
- Operators

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

What is a Tensor?

- A tensor is an array that represents the types of data.
- A tensor can have *n* dimensions
- In a tensor, the values hold identical data types with a known shape.
- The shape represents dimensionality.
- Thus, a vector will be a one-dimensional tensor, a matrix is a two-dimensional tensor, and a scalar would be a zero-dimensional tensor.





Example:

Image → 3D Tensor (Height × Width × Channels)

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Shape

- In the TensorFlow Python library, shape refers to the dimensionality of the tensor.

- So, the shape of the tensor is (2,2,2).

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Data structures in tensor flow

- In TensorFlow, data is stored and processed using **Tensors**.
- A tensor is a multi-dimensional array used to represent numerical data.
- Simple Idea
  - Tensor = Container that holds data
  - Used as input, intermediate data, and output in models
- Types of Tensor Data Structures
  - Scalar (0D Tensor)
  - Vector (1D Tensor)
  - Matrix (2D Tensor)
  - 3D Tensor and Higher

Type	Example	Dimension
Scalar	Single number	0D
Vector	List of numbers	1D
Matrix	Table of numbers	2D
Tensor	Multi-dimensional data	3D+

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Other TensorFlow Data Structures

Variables

- Used to store model parameters
- Values change during training

Constants

- Fixed values
- Do not change

Sparse Tensors

- Efficient storage for large sparse data

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Type

The type represents the kind of data that the values in a tensor hold.

All values in a tensor hold an identical data type.

Datatypes in TensorFlow

- integers
- floating point
- unsigned integers
- booleans
- strings
- integer with quantized ops
- complex numbers

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Graph – Session - Operators

Graph

- A graph is a set of computations that take place successively on input tensors. It comprises an arrangement of nodes representing the mathematical operations in a model.

Session

- A session in TensorFlow executes the operations in the graph. It is run to evaluate the nodes in a graph.

Operators

- Operators in TensorFlow are pre-defined mathematical operations.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Components of TensorFlow

Tensors

- Store data
- Similar to NumPy arrays
- Used as inputs and outputs

Operations (Ops)

- Mathematical computations
- Example: Addition, Multiplication, Convolution

Computational Graph

- Represents flow of data
- Nodes → Operations
- Edges → Data (Tensors)

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

TensorFlow Workflow

Step 1: Import Data

Step 2: Define Model Architecture

Step 3: Train Model

Step 4: Evaluate Model

Step 5: Deploy Model

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

tensorflow (Core Library)

- **Purpose**
  - Core numerical computation library
  - Supports **tensor operations**, **automatic differentiation**, and **hardware acceleration**
- **Key Functionalities**
  - Tensor creation (tf.constant, tf.Variable)
  - Mathematical operations
  - Computational graph execution
  - GPU/TPU support
- Example: Creating Tensors

```
import tensorflow as tf
a = tf.constant([1, 2, 3])
b = tf.constant([4, 5, 6])
print(a + b)
```

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

tensorflow.keras (High-Level API)

- **Purpose**
  - Simplifies **deep learning model development**
  - Recommended API for most users
- **Submodules**
  - models – model containers
  - layers – neural network layers
  - optimizers – Adam, SGD, RMSProp
  - losses – MSE, Cross-Entropy
  - callbacks – EarlyStopping, ModelCheckpoint

```
from tensorflow.keras import layers, models
model = models.Sequential([
    layers.Dense(64, activation='relu'),
    layers.Dense(1)
])
```

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

tensorflow.data (Data Pipeline Library)

- **Purpose**
  - Efficient **data loading, preprocessing, and batching**
  - Handles large datasets without loading into memory
- **Features**
  - Data streaming
  - Parallel loading
  - Prefetching
- **Example**

```
dataset = tf.data.Dataset.from_tensor_slices(([1,2,3],[4,5,6]))
dataset = dataset.batch(2)
```

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

tensorflow.nn (Neural Network Operations)

- **Purpose**
  - Low-level neural network functions
  - Used internally by Keras and advanced users
- **Includes**
  - Activation functions
  - Convolution operations
  - Pooling functions
- **Example**
  - tf.nn.relu([-1.0, 2.0, -3.0])

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

tensorflow.optimizers

- **Purpose**
  - Implements **optimization algorithms**
  - Updates model weights during training
- **Common Optimizers**
  - SGD
  - Adam
  - RMSProp
  - Adagrad
- **Example**

```
optimizer = tf.keras.optimizers.Adam(learning_rate=0.001)
```

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

tensorflow.losses

- **Purpose**
  - Measures **prediction error**
  - Guides weight updates during training
- **Common Loss Functions**
  - Mean Squared Error (Regression)
  - Binary Crossentropy
  - Categorical Crossentropy
- **Example**

```
loss_fn = tf.keras.losses.MeanSquaredError()
```

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

tensorflow.metrics

- **Purpose**
  - Evaluates **model performance**
  - Does NOT affect training directly
- **Examples**
  - Accuracy
  - Precision
  - Recall
- **Example**

```
metric = tf.keras.metrics.Accuracy()
```

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

tensorflow.image

- **Purpose**
  - Image preprocessing and augmentation
- **Operations**
  - Resize
  - Flip
  - Rotate
  - Normalize
- **Example**

tf.image.flip_left_right(image)

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

tensorflow.random

- **Purpose**
  - Random number generation
  - Used for initialization and augmentation
- **Example**

tf.random.normal(shape=(3,3))

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

tensorflow.io

- **Purpose**
  - File input/output operations
  - Reads images, CSVs, TFRecords
- **Example**

```
tf.io.read_file("data.txt")
```

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

tensorflow.saved_model

- **Purpose**
  - Save and load trained models
  - Used for deployment
- **Example**
  - model.save("my_model")

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

TensorFlow + Keras Workflow

- **Standard Pipeline**
  - Load data
  - Build model
  - Compile model
  - Train model
  - Evaluate & predict

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Building a Neural Network

- **Step 1: Import Libraries**

```
import tensorflow as tf
from tensorflow.keras import layers, models
```

- **Step 2: Prepare Data**

```
import numpy as np
X = np.array([[0,0],[0,1],[1,0],[1,1]])
y = np.array([0,1,1,0])   # XOR problem
```

- **Step 3: Create the Model**

```
model = models.Sequential([
    layers.Dense(8, activation='relu', input_shape=(2,)),
    layers.Dense(4, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])
```

`Dense` → fully connected layer
`ReLU` → non-linear learning
`Sigmoid` → binary output

This file is meant for personal use by [illegible]
Sharing or publishing the contents in part or full is liable for legal action.

- **Step 4: Compile the Model**

```
model.compile( optimizer='adam’,   loss='binary_crossentropy’,    metrics=['accuracy'])
```

  - **Optimizer**: updates weights
  - **Loss**: measures error
  - **Metrics**: performance tracking

- **Step 5: Train the Model**

```
model.fit(X, y, epochs=1000, verbose=0)
```

Internally Happens

Forward propagation
Loss calculation
Backpropagation
Weight updates

- **Step 6: Evaluate and Predict**

```
loss, acc = model.evaluate(X, y)
print("Accuracy:", acc)
pred = model.predict(X)
print(pred)
```

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Regularization in TensorFlow

- **Dropout**

```
layers.Dropout(0.5)
```

- **L2 (Ridge)**

```
layers.Dense(64, kernel_regularizer=tf.keras.regularizers.l2(0.01))
```

- **Batch Normalization**

```
layers.BatchNormalization()
```

- **Early Stopping**

```
callback = tf.keras.callbacks.EarlyStopping(
    monitor='val_loss',
    patience=5
)
```

- Stops training automatically
- Prevents overfitting

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Model Summary & Visualization

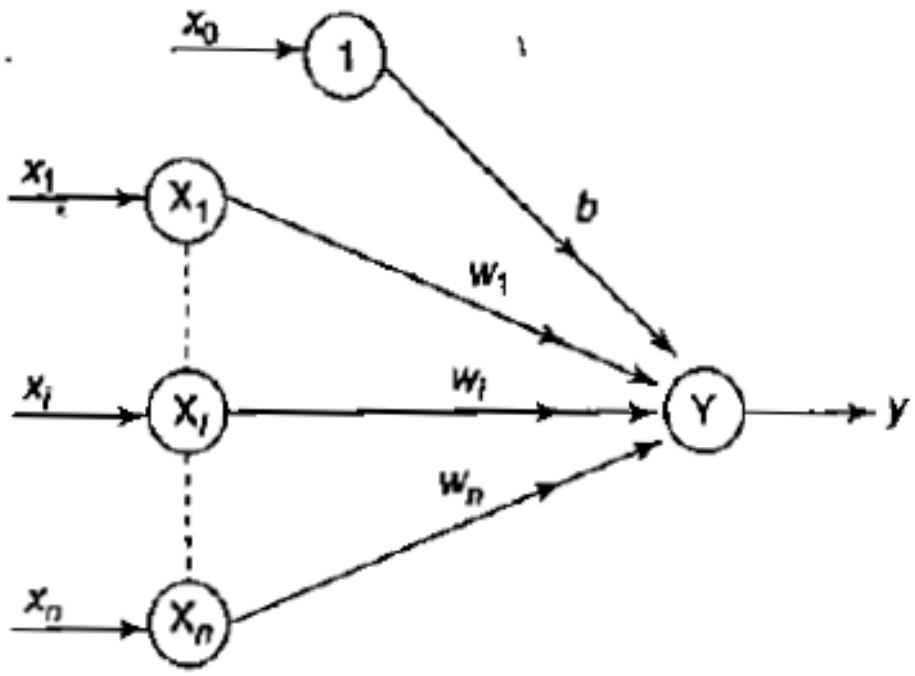
model.summary()

- Shows:
  - Layers
  - Parameters
  - Output shapes

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Perceptron Model

- Single-layer feed-forward networks
- Also called simple perceptron's
- It is used to classify the input pattern as a member or not a member to a particular class
- It consist of n input neurons, 1 output neuron and a bias
- Input and output neurons are connected through a directed communication link



This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Perceptron Learning Rule

- Consist of three units – sensory unit, associator unit, response unit
- The response unit has an activation of 1,0 or -1
- The output is given by

  $y=f(y_{in})$

$$f(y_{in}) = \begin{cases} 1 & \text{if } y_{in} > \theta \\ 0 & \text{if } -\theta \le y_{in} \le \theta \\ -1 & \text{if } y_{in} < -\theta \end{cases}$$

- The weights will be adjusted if error occurs

$$w_i(\text{new}) = w_i(\text{old}) + \alpha t x_i$$
$$b(\text{new}) = b(\text{old}) + \alpha t$$

where t is +1 or -1 and α is learning rate

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Learning Ex-OR problem

- XOR is a non-linear problem
- Single perceptron cannot solve XOR
- Multi-layer neural networks solve XOR

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Problem-4

X1	X2	Y
0	0	0
0	1	1
1	0	1
1	1	0

Implement XOR function using Multi Layer Network

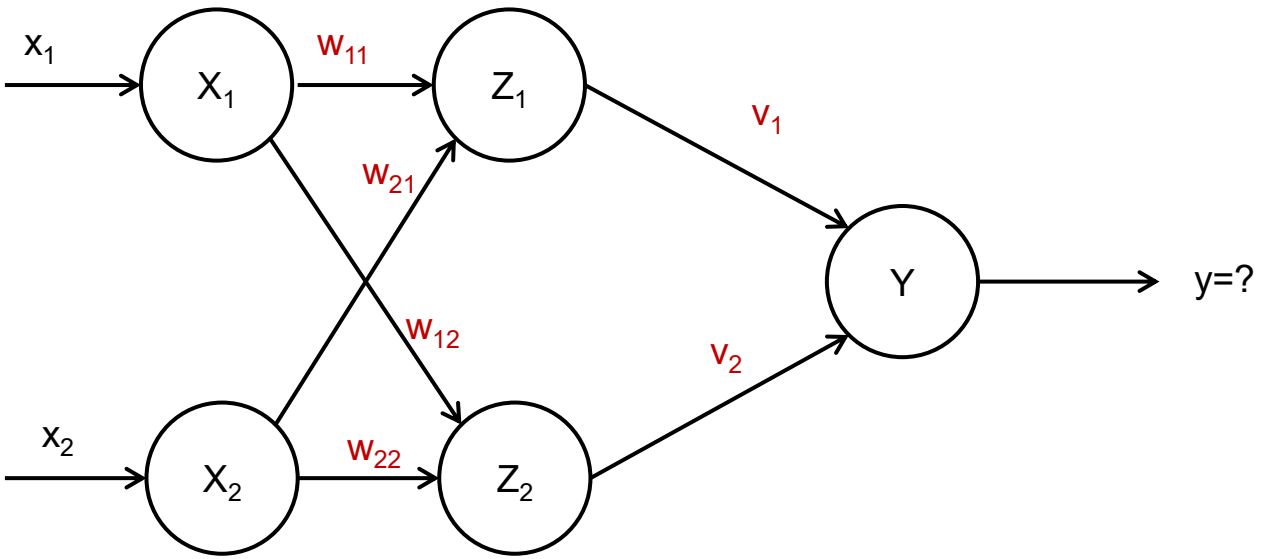
Solution:

The truth table for XOR function is,

XOR function cannot be represented by single logic function. It is represented as,

$y = X_1\bar{x}_2 + \bar{x}_1 X_2$

$y = z_1 + z_2$



This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Step-1: Calculate Z_1

- **Iteration-1:**

  Assume both weights as 1(excitatory).

  $w_{11}=w_{21}=1$

Calculate net inputs.

$x_1=0, x_2=0 \Rightarrow z_{1in} = w_{11}.x_1 + w_{21}.x_2 = 0 \times 1 + 0 \times 1 = 0 + 0 = 0$

$x_1=0, x_2=1 \Rightarrow z_{1in} = w_{11}.x_1 + w_{21}.x_2 = 0 \times 1 + 1 \times 1 = 0 + 1 = 1$

$x_1=1, x_2=0 \Rightarrow z_{1in} = w_{11}.x_1 + w_{21}.x_2 = 1 \times 1 + 0 \times 1 = 1 + 0 = 1$

$x_1=1, x_2=1 \Rightarrow z_{1in} = w_{11}.x_1 + w_{21}.x_2 = 1 \times 1 + 1 \times 1 = 1 + 1 = 2$

Hence, it is not possible to get expected output using these weights

- **Iteration-2:**

  Assume, $w_{11}=1$, $w_{21}=-1$

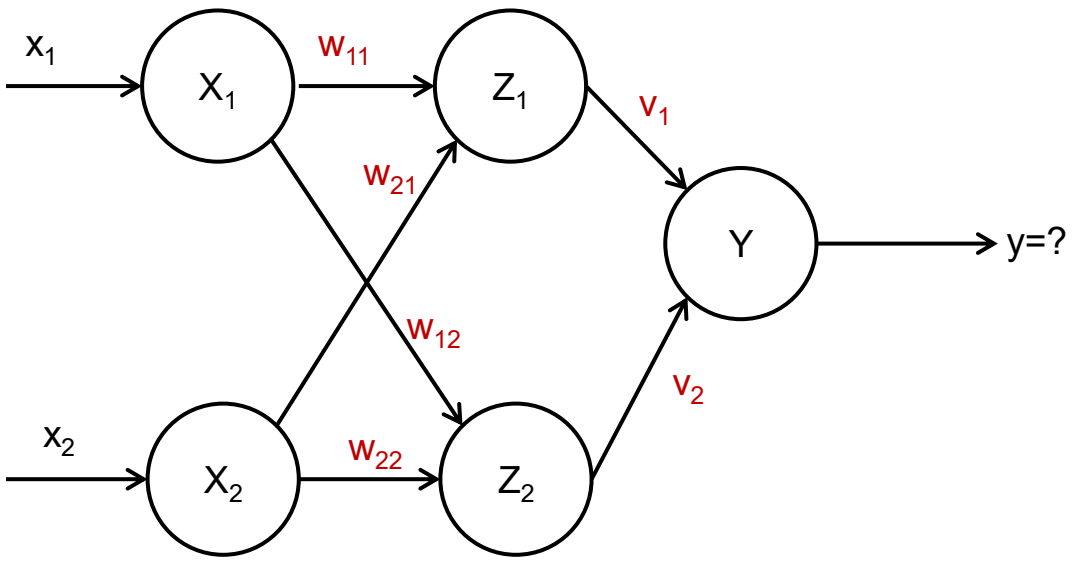
Calculate net inputs,

$x_1=0, x_2=0 \Rightarrow z_{1in} = w_{11}.x_1 + w_{21}.x_2 = 0 \times 1 + 0 \times -1 = 0$

$x_1=0, x_2=1 \Rightarrow z_{1in} = w_{11}.x_1 + w_{21}.x_2 = 0 \times 1 + 1 \times -1 = -1$

$x_1=1, x_2=0 \Rightarrow z_{1in} = w_{11}.x_1 + w_{21}.x_2 = 1 \times 1 + 0 \times -1 = 1$

$x_1=1, x_2=1 \Rightarrow z_{1in} = w_{11}.x_1 + w_{21}.x_2 = 1 \times 1 + 1 \times -1 = 0$



On the basis of this calculated net input, it is possible to get the required output.

Hence, $w_{11}=1$, $w_{21}=-1$

$\theta \geq 1$ for Z_1 neuron

$x_1=0, x_2=0$	$\Rightarrow$	$z_1 = 0$
$x_1=0, x_2=1$	$\Rightarrow$	$z_1 = 0$
$x_1=1, x_2=0$	$\Rightarrow$	$z_1 = 1$
$x_1=1, x_2=1$	$\Rightarrow$	$z_1 = 0$

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Step-2: Calculate Z_2

- Iteration-1:

  Assume both weights as 1(excitatory). $w_{12}=w_{22}=1$

  Calculate net inputs.

  $x_1=0, x_2=0 \Rightarrow z_{2in} = w_{12}.x_1 + w_{22}.x_2 = 1 \times 0 + 1 \times 0 = 0$

  $x_1=0, x_2=1 \Rightarrow z_{2in} = w_{12}.x_1 + w_{22}.x_2 = 1 \times 0 + 1 \times 1 = 1$

  $x_1=1, x_2=0 \Rightarrow z_{2in} = w_{12}.x_1 + w_{22}.x_2 = 1 \times 1 + 1 \times 0 = 1$

  $x_1=1, x_2=1 \Rightarrow z_{2in} = w_{12}.x_1 + w_{22}.x_2 = 1 \times 1 + 1 \times 1 = 2$

Hence, it is not possible to get expected output using these weights

- Iteration-2:

  Assume, $w_{12}=-1$, $w_{22}=1$

  Calculate net inputs,

  $x_1=0, x_2=0 \Rightarrow z_{2in} = w_{12}.x_1 + w_{22}.x_2 = 0 \times -1 + 0 \times 1 = 0$

  $x_1=0, x_2=1 \Rightarrow z_{2in} = w_{12}.x_1 + w_{22}.x_2 = 0 \times -1 + 1 \times 1 = 1$

  $x_1=1, x_2=0 \Rightarrow z_{2in} = w_{12}.x_1 + w_{22}.x_2 = 1 \times -1 + 0 \times 1 = -1$

  $x_1=1, x_2=1 \Rightarrow z_{2in} = w_{12}.x_1 + w_{22}.x_2 = 1 \times -1 + 1 \times 1 = 0$

On the basis of this calculated net input, it is possible to get the required output.

Hence, $w_{21}=-1$, $w_{22}=1$

$\theta \geq 1$

$x_1=0, x_2=0$	$\Rightarrow$	$z_2 = 0$
$x_1=0, x_2=1$	$\Rightarrow$	$z_2 = 1$
$x_1=1, x_2=0$	$\Rightarrow$	$z_2 = 0$
$x_1=1, x_2=1$	$\Rightarrow$	$z_2 = 0$

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Step-3: Calculate, $y=Z_1$ OR Z_2

$$y_{in}=z_1v_1 + z_2v_2$$

- Iteration-1:

Assume both weights as 1(excitatory).

$$v_1=v_2=1$$

$z_1=0, z_2=0 \Rightarrow \quad z_{1in} = v_1.z_1+ v_2.z_2 = 1 \times 0 + 1 \times 0 = 0, y = 0$

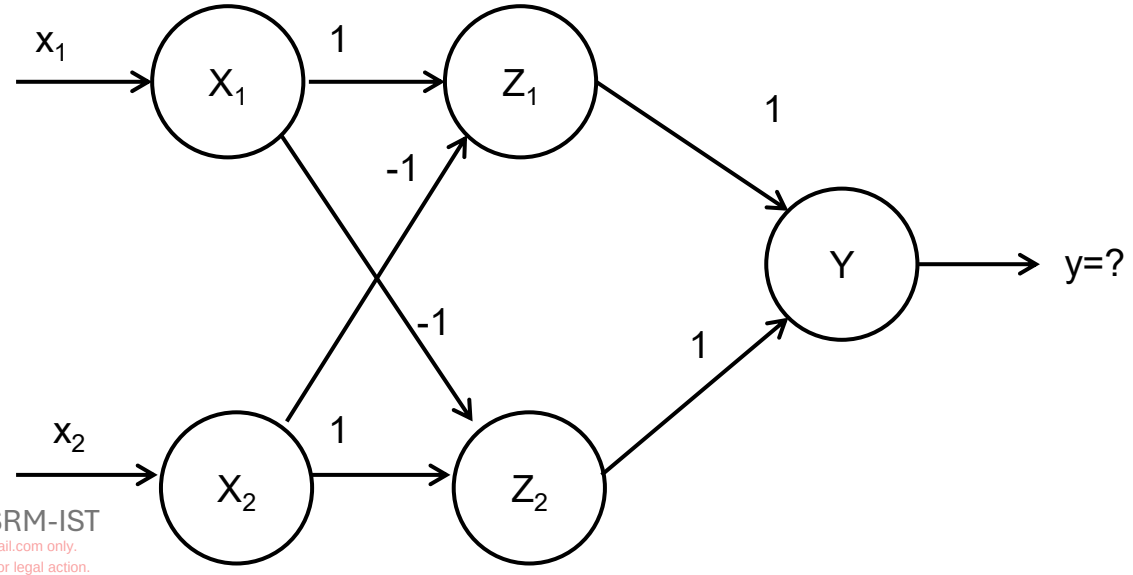
$z_1=0, z_2=1 \Rightarrow \quad z_{1in} = v_1.z_1+ v_2.z_2 = 1 \times 0 + 1 \times 1 = 1, y =1$

$z_1=1, z_2=0 \Rightarrow \quad z_{1in} = v_1.z_1+ v_2.z_2 = 1 \times 1 + 1 \times 0 = 1, y = 1$

$z_1=0, z_2=0 \Rightarrow \quad z_{1in} = v_1.z_1+ v_2.z_2 = 1 \times 0 + 1 \times 0 = 0, y =0$

Setting threshold of $\theta \geq 1$ res, $v_1=v_2=1$ and $w_{11}=w_{22}=1, w_{12}=w_{21}=-1$
the net is recognized

x_1	x_2	z_1	z_2	y
0	0	0	0	0
0	1	0	1	1
1	0	1	0	1
1	1	0	0	0



This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Activation functions

- Introduce non-linearity
- Types: Sigmoid, ReLU, Tanh, Softmax
- Helps neural network learn complex patterns

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

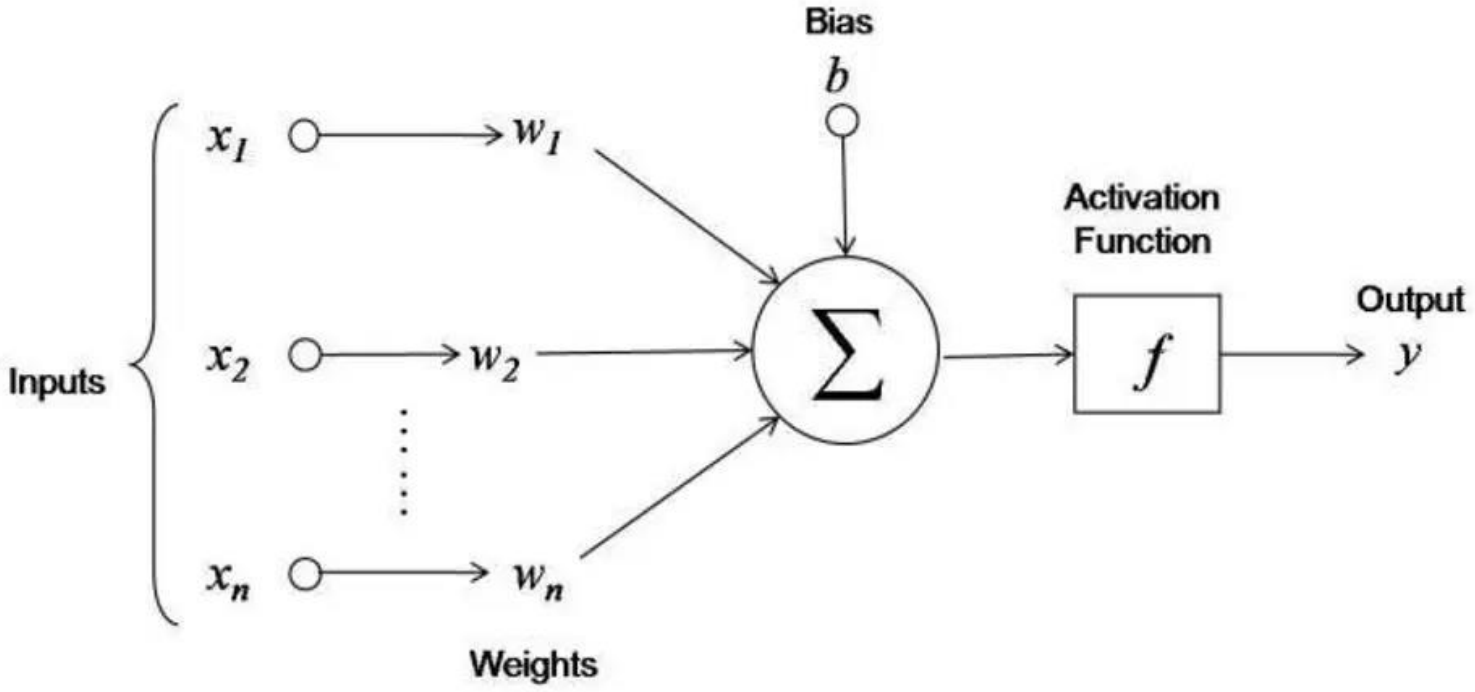
Activation Function

- **Activation function** of a node defines the output of that node given an input or set of inputs
- It is also called the **Transfer Function**
- **It decides whether it should be "fired" or not**

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Role of activation function

- An activation function determines whether a neuron should be activated and how strongly its output should influence the next layer.
- It decides what information passes forward and what gets suppressed.



This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Types of Activation Functions

The most famous activation functions are given below,

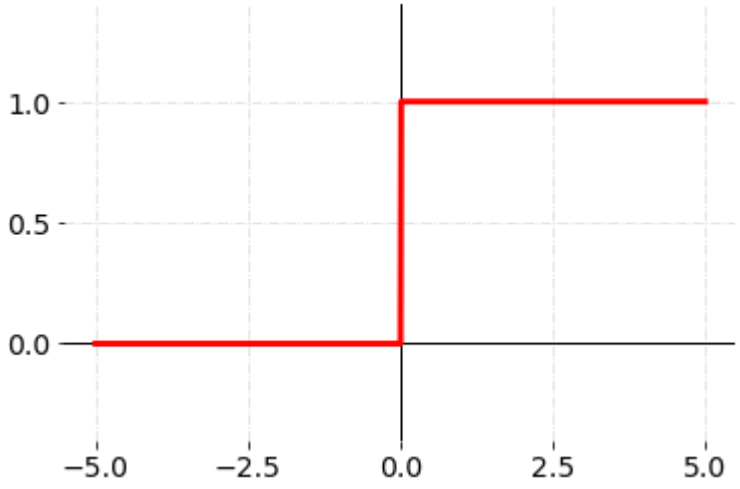
- Binary step
- Linear
- Sigmoid
- Tanh(Hyperbolic Tangent)
- ReLU(Rectified Linear unit)
- LeakyReLU
- Softmax

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Binary step

- The Binary Step function activates a neuron by producing a binary output (0 or 1) based on a threshold.
- Equation

$$f(x) = \begin{cases} 1 & x \geq 0 \\ 0 & x < 0 \end{cases}$$



Flat line at 0 for negative inputs
Jumps suddenly to 1 at threshold

- Basic ON/OFF Control System
  - Motion sensor
    - Motion detected → 1
    - No motion → 0
- When to Use
  - Simple rule-based systems
  - Early perceptron models

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Binary Step Activation

- **Scenario: Automatic Street Light System**
  - Sensor measures light intensity
  - If intensity ≥ threshold → **Light ON (1)**
  - Else → **Light OFF (0)**
- Why Binary Step?
  - Only **two states**
  - No need for probabilities
  - Simple decision logic
- Not suitable for learning-based systems because output does not change smoothly.

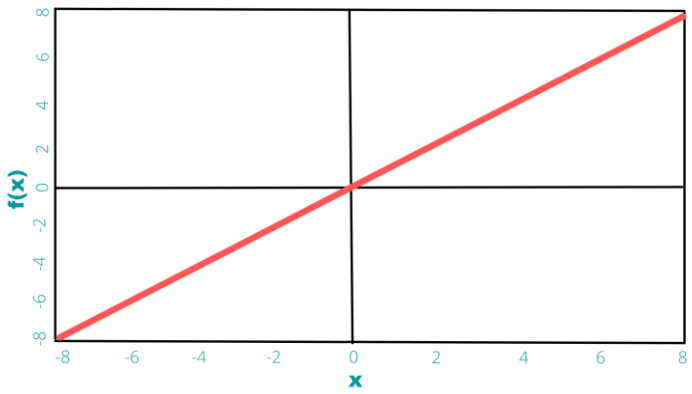
This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Linear (Identity) Activation Function

- Outputs the input as it is, without modification.
- Equation

$$f(x) = x$$

- Real-Time Scenario
  - House Price Prediction
  - Output = ₹45,00,000
  - Continuous numeric value
- When to Use
  - Output layer of regression models



Linear Activation Function

Straight line through origin
No non-linearity

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Linear Activation – Continuous Value Prediction

- Scenario: House Price Prediction
  - Inputs: area, location score, age of building
  - Output: ₹ price (continuous)
  - Example:
  - Model predicts ₹48,50,000
- Why Linear?
  - Output must be unbounded
  - No squashing of values
  - Used in regression output layer

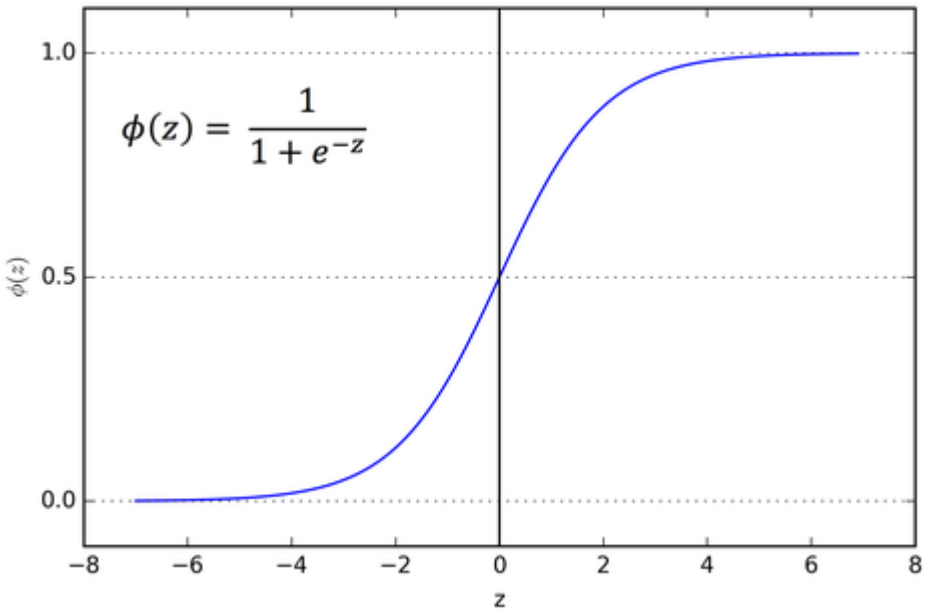
This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Sigmoid Activation Function

- Maps input into (0, 1) range, useful for probability estimation.
- Equation

$$f(x) = \frac{1}{1 + e^{-x}}$$

- Real-Time Scenario
  - Spam Email Detection
  - Output = 0.94 → 94% spam
  - Output = 0.06 → Not spam
- When to Use
  - Binary classification output layer



S-shaped curve
Saturates at 0 and 1
Causes **vanishing gradients**

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Sigmoid Activation – Binary Probability Output

- Scenario: Medical Diagnosis System
  - Predicts whether a patient has diabetes
- Output:
  - 0.87 → 87% probability of disease
  - Doctor applies threshold (e.g., 0.5)
- Why Sigmoid?
  - Output lies between 0 and 1
  - Interpretable as probability

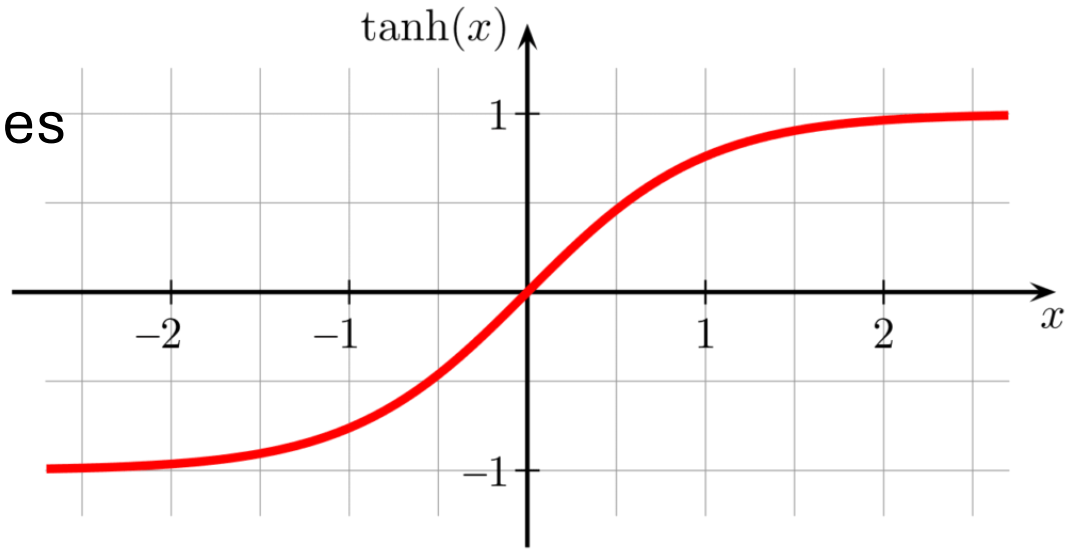
This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Tanh (Hyperbolic Tangent) Activation Function

- Zero-centered sigmoid variant producing values between −1 and +1.
- Equation

$$f(x) = tanh(x)$$

- Real-Time Scenario
  - Sentiment Analysis
    - −1 → Negative
    - 0 → Neutral
    - +1 → Positive
- When to Use
  - Shallow neural networks
  - Early RNN architectures



S-shaped
Centered at zero
Still suffers from vanishing gradient

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Tanh Activation – Balanced Signal Modeling

- Scenario: Sentiment Analysis (Text Reviews)
- Output range:
  - −1 → Negative
  - 0 → Neutral
  - +1 → Positive
- Example:
  - Review score = +0.72 → Strong positive sentiment
- Why Tanh?
  - Zero-centered outputs
  - Better than sigmoid for hidden layers in shallow networks

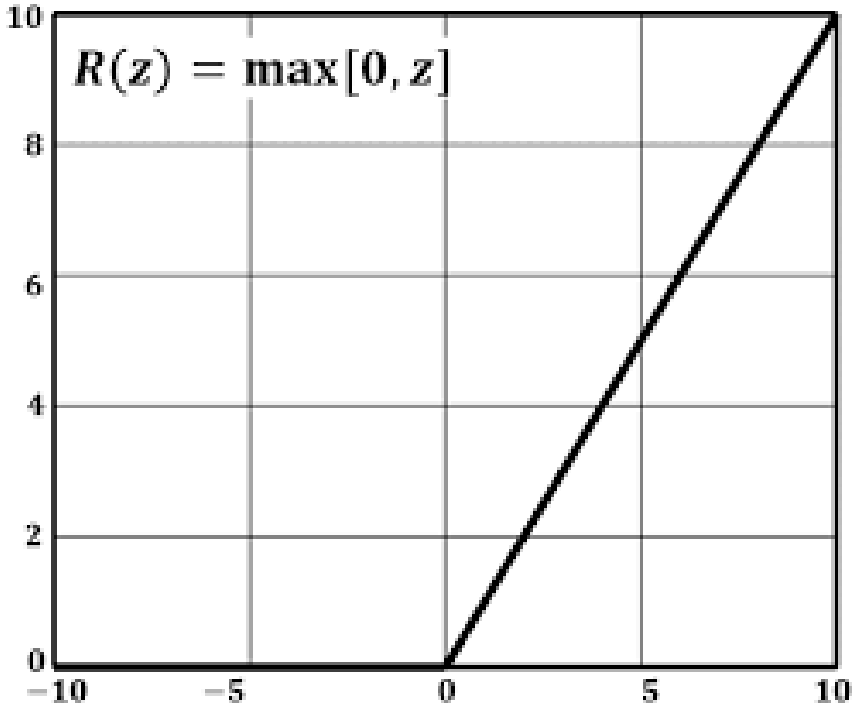
This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

ReLU (Rectified Linear Unit)

- Outputs zero for negative values and linear for positive values.
- Equation

$$f(x) = max(0, x)$$

- Real-Time Scenario
  - Image Recognition (CNN)
    - Edge detected → positive value passed
    - Noise → suppressed to zero
- When to Use
  - Hidden layers in CNNs & DNNs
  - Default choice in deep learning



Flat at 0 for negatives
Linear growth for positives
Sparse activation

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

ReLU Activation – Feature Extraction (MOST COMMON)

- Scenario: Face Recognition System
- CNN detects:
  - Edges
  - Eyes
  - Nose
  - Face contours
- Behavior:
  - Strong feature → passed forward
  - Weak/noisy feature → set to 0
- Why ReLU?
  - Fast computation
  - Sparse activation
  - Prevents vanishing gradient

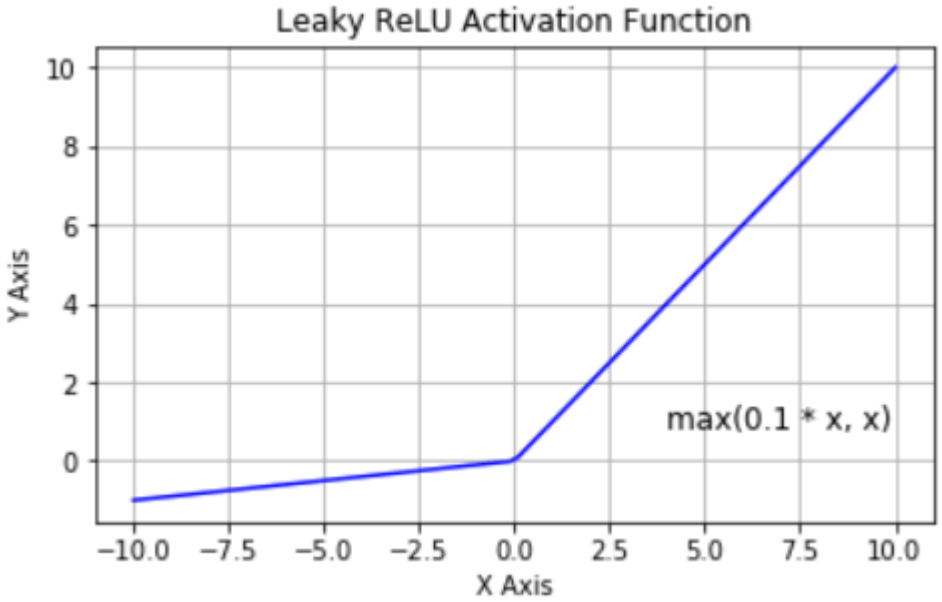
This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Leaky ReLU

- Improved ReLU that allows a small gradient for negative inputs.
- Equation

$$f(x) = \begin{cases} x & x > 0 \\ 0.01x & x \leq 0 \end{cases}$$

- Real-Time Scenario
  - Medical Imaging (Tumor Detection)
  - Weak signals still carry diagnostic information
- When to Use
  - Deep networks where ReLU neurons die
  - Vision and healthcare models



Small negative slope instead of zero
Prevents neuron death

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Leaky ReLU – Weak Signal Preservation

- Scenario: Medical Image Analysis (MRI / CT Scan)
  - Tumor edges may appear faint
  - Weak signals are still meaningful
- Behavior:
  - Negative inputs → small negative output instead of 0
- Why Leaky ReLU?
  - Prevents neuron death
  - Ensures gradient flow for all neurons

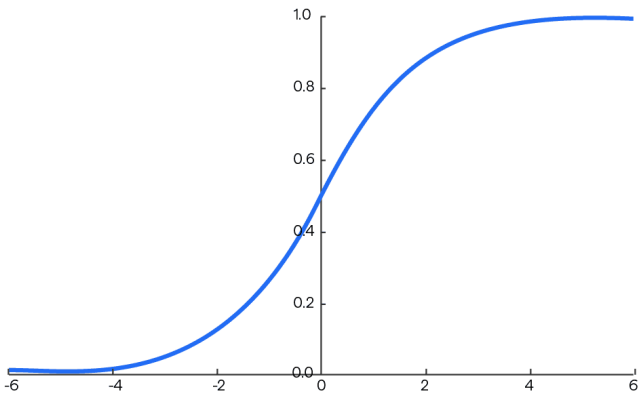
This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Softmax Activation Function

- Converts logits into a probability distribution over multipl
- Equation

$$f(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

- Real-Time Scenario
  - Handwritten Digit Recognition
    - Digit 7 → 0.96
    - Other digits → near zero
- When to Use
  - Multiclass classification output layer



Outputs sum to 1
Highlights dominant class

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Softmax Activation – Multiclass Decision Making

- Scenario: Handwritten Digit Recognition
- Model predicts digit from 0–9
- Output:
  - 0 → 0.02
  - 3 → 0.05
  - 7 → 0.91
- Why Softmax?
  - Converts scores into probability distribution
  - Ensures probabilities sum to 1
  - Highlights most likely class

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Example

- Let:
  - Input $x = 2$
  - Weight $w = 0.6$
  - Bias $b = -0.5$
- First compute **net input**:

$$z = wx + b = (0.6)(2) - 0.5 = 1.2 - 0.5 = 0.7$$

- We now apply different activation functions on **z = 0.7**.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Binary Step Activation Function

- **Equation**

$$f(z) = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0 \end{cases}$$

- **Numerical Calculation**

$$z = 0.7 \geq 0 \Rightarrow f(z) = 1$$

- **Interpretation (Real-Time)**
  - Motion detected
  - Loan approved
  - Spam email

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Linear (Identity) Activation Function

- Equation

$$f(z) = z$$

- Numerical Calculation

$$f(0.7) = 0.7$$

- Interpretation (Real-Time)
  - Predicted temperature = 0.7 (scaled)
  - House price = ₹7 lakhs (after scaling)

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Sigmoid Activation Function

- **Equation**

$$f(z) = \frac{1}{1 + e^{-z}}$$

- **Numerical Calculation**

$$f(0.7) = \frac{1}{1 + e^{-0.7}}$$

$$e^{-0.7} \approx 0.4966$$

$$f(0.7) = \frac{1}{1 + 0.4966} \approx 0.668$$

- **Interpretation (Real-Time)**
  - 66.8% probability of **spam**
  - 66.8% chance of **disease present**

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Tanh (Hyperbolic Tangent)

- Equation

$$f(z) = \tanh(z)$$

- Numerical Calculation

$$\tanh(0.7) \approx 0.604$$

- Interpretation (Real-Time)
  - +0.604 → Moderately positive sentiment

  - −1 → Strong negative
  - +1 → Strong positive

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

ReLU (Rectified Linear Unit)

- Equation

$$f(z) = \max(0, z)$$

- Numerical Calculation

$$f(0.7) = \max(0, 0.7) = 0.7$$

- If z were negative (example: z = −0.4):

$$f(-0.4) = 0$$

- Interpretation (Real-Time)
  - Important feature → passed forward
  - Noise → suppressed

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Leaky ReLU

- **Equation**

$$f(z) = \begin{cases} z & z > 0 \\ 0.01z & z \leq 0 \end{cases}$$

- **Numerical Calculation**

For $z = 0.7$:

$$f(0.7) = 0.7$$

For $z = -0.4$:

$$f(-0.4) = 0.01(-0.4) = -0.004$$

- **Interpretation (Real-Time)**
  - Weak but useful features are retained
  - Prevents neuron death

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Softmax Activation Function (Multiclass)

- **Scenario:** Digit classification (0, 1, 2)
- Let logits:

$$z = [2.0, \ , 1.0 \ \ 0.1]$$

- **Equation**

$$f(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

- **Numerical Calculation**

$$e^2 = 7.39, e^1 = 2.71, e^{0.1} = 1.105$$

$$\text{Sum} = 7.39 + 2.71 + 1.105 = 11.205$$

$$P(0) = \frac{7.39}{11.205} = 0.66 \qquad P(1) = \frac{2.71}{11.205} = 0.24 \qquad P(2) = \frac{1.105}{11.205} = 0.10$$

- **Interpretation (Real-Time)**
  - Digit predicted = **0** (highest probability 66%)

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Python Program Example for Activation Functions

```
import tensorflow as tf
# Inputs, weights, bias
x = tf.constant([1.5, -2.0])
w = tf.constant([0.8, 0.4])
b = 0.5

# Step 1: Weighted sum
z = tf.reduce_sum(x * w) + b
print("Weighted sum (z):", z.numpy())

# Step 2: Apply activation functions
print("Binary Step:", tf.where(z >= 0, 1.0, 0.0).numpy())
print("Linear:", tf.keras.activations.linear(z).numpy())
print("Sigmoid:", tf.keras.activations.sigmoid(z).numpy())
print("Tanh:", tf.keras.activations.tanh(z).numpy())
print("ReLU:", tf.keras.activations.relu(z).numpy())

leaky_relu = tf.keras.layers.LeakyReLU(alpha=0.01)
print("Leaky ReLU:", leaky_relu(z).numpy())

# Softmax example
z_softmax = tf.constant([0.9, 1.2, -0.5])
softmax_output = tf.keras.activations.softmax(z_softmax)
print("Softmax:", softmax_output.numpy())
```

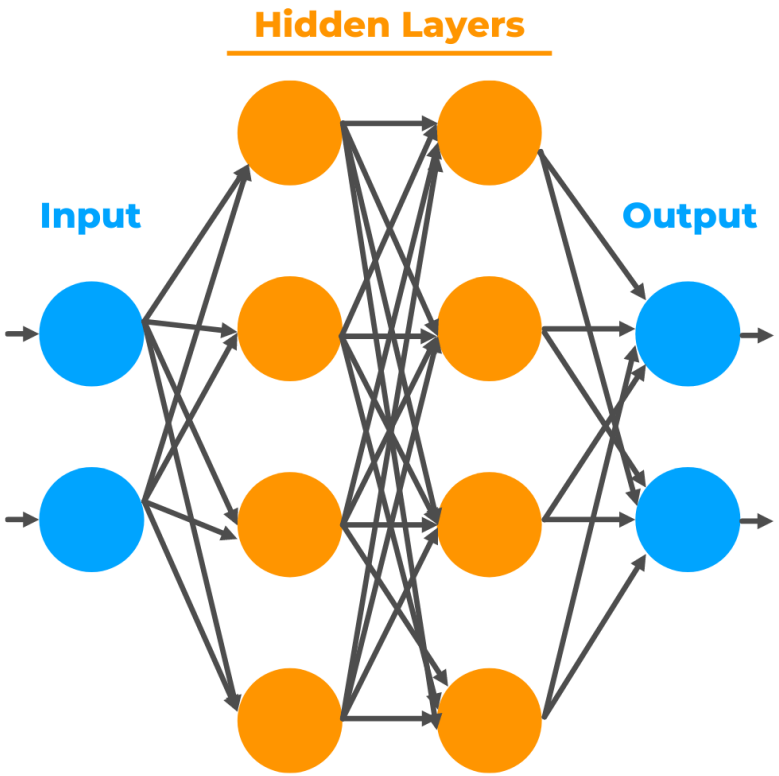
This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

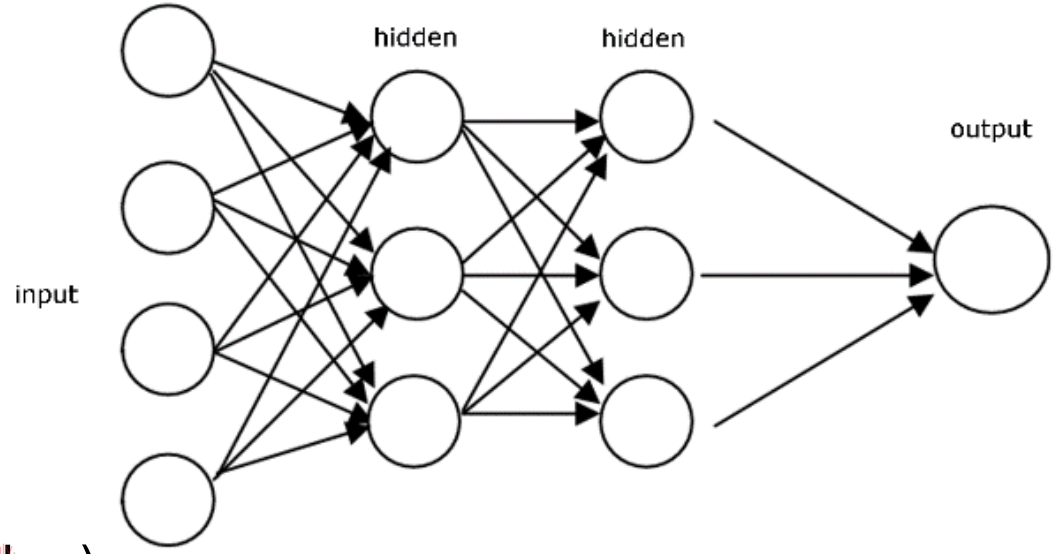
Feedforward neural networks

- A Feedforward Neural Network is the simplest type of artificial neural network where information flows in one direction only:

  Input → Hidden layer(s) → Output

- There are no loops or feedback connections. Each layer passes its output forward to the next layer.
- **Input Layer**
  - Receives raw features (no computation, just passes values).
- **Hidden Layer(s)**
  - Perform learning via:
    - weighted sum
    - bias addition
    - activation function
- **Output Layer**
  - Produces final prediction (class, probability, or value).





This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in any form is prohibited. Full access for 365 days.

Feedforward neural networks (FFNN)

- **Mathematical Model (Single Neuron)**
- For neuron j:

$$z_j = \sum_i w_{ij}\, x_i + b_j, a_j = f(z_j)$$

- Where:

  x_i :inputs

  w_{ij} :weights

  b_j :bias

  $f(\cdot)$:activation function

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

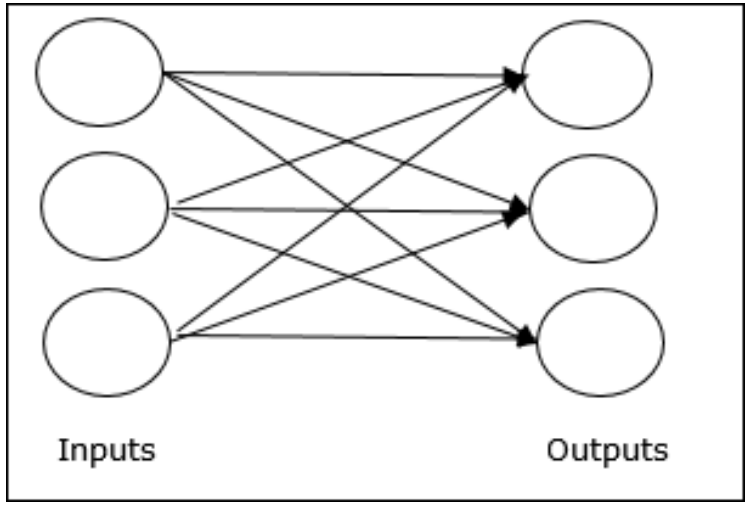
Feedforward neural networks (FFNN)

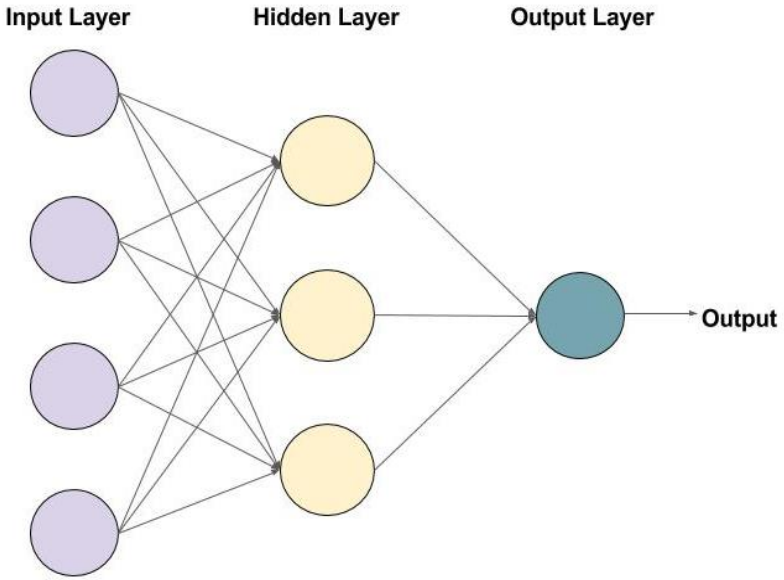
- Forward Propagation
  - Inputs are multiplied by weights
  - Bias is added
  - Activation function is applied
  - Output flows to next layer
- Loss Computation
  - Compare prediction with ground truth
  - Compute error using a loss function (e.g., MSE, Cross-Entropy)
- Backpropagation (Training Phase)
  - Error gradients flow backward
  - Weights are updated using Gradient Descent
- Note: The network structure is feedforward, but training uses backpropagation.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Types of Feedforward Networks

- Single-Layer Perceptron
  - No hidden layer
  - Solves only linearly separable problems
- Multilayer Perceptron (MLP)
  - One or more hidden layers
  - Uses non-linear activation functions
  - Can solve complex problems
  - Example:Back Propagation Network





This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Activation Functions Used

- **Hidden layers**
  - ReLU / Leaky ReLU / Tanh
- **Output layer**
  - Binary classification → Sigmoid
  - Multiclass classification → Softmax
  - Regression → Linear

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Real-Time Example: Student Performance Prediction

- **Inputs**
  - Study hours
  - Attendance %
  - Assignment score
- **Hidden Layers**
  - Learn patterns like: consistency, effort, learning pace
- **Output**
  - Pass / Fail (Sigmoid)
    OR
  - Grade (A/B/C) (Softmax)
    OR
  - Final marks (Linear)

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Python Program: Feedforward Neural Network (TensorFlow)

- Problem: Binary Classification (Pass / Fail)
- **Inputs:**
  - Study hours
  - Attendance %
- **Output:**
  - 1 → Pass
  - 0 → Fail

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

```python
import tensorflow as tf
import numpy as np

# ------------------------------
# Step 1: Create Dataset
# ------------------------------
# Features: [Study Hours, Attendance %]
X = np.array([
    [1, 45],
    [2, 50],
    [4, 65],
    [6, 75],
    [8, 85],
    [9, 95]
], dtype=float)

# Output: Pass (1) / Fail (0)
y = np.array([0, 0, 0, 1, 1, 1], dtype=float)
```

```python
# ------------------------------
# Step 2: Model Definition Function
# ------------------------------
def create_model():
    model = tf.keras.Sequential([
        tf.keras.layers.Dense(8, activation='relu', input_shape=(2,)),
        tf.keras.layers.Dense(4, activation='relu'),
        tf.keras.layers.Dense(1, activation='sigmoid')
    ])


model.compile(
        optimizer='adam',
        loss='binary_crossentropy',
        metrics=['accuracy']
    )
    return model
```

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

```
# ------------------------------
# Step 3: Training WITHOUT Normalization
# ------------------------------
print("\nTraining WITHOUT Normalization")
model_without_norm = create_model()
history_without = model_without_norm.fit( X, y, epochs=50, verbose=0 )

loss_wo, acc_wo = model_without_norm.evaluate(X, y, verbose=0)
print("Accuracy without normalization:", acc_wo)

# ------------------------------
# Step 4: Apply Normalization
# ------------------------------
X_normalized = X / X.max(axis=0)

print("\nNormalized Input Data:")
print(X_normalized)
```

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

```
# ------------------------------
# Step 5: Training WITH Normalization
# ------------------------------
print("\nTraining WITH Normalization")
model_with_norm = create_model()
history_with = model_with_norm.fit(
    X_normalized, y, epochs=50, verbose=0
)
loss_w, acc_w = model_with_norm.evaluate(X_normalized,
y, verbose=0)
print("Accuracy with normalization:", acc_w)
```

```
# ------------------------------
# Step 6: Prediction Comparison
# ------------------------------
test_input = np.array([[7, 80]], dtype=float)

# Prediction without normalization
pred_wo = model_without_norm.predict(test_input)

# Prediction with normalization
test_input_norm = test_input / X.max(axis=0)
pred_w = model_with_norm.predict(test_input_norm)

print("\nTest Input: Study Hours = 7, Attendance = 80%")
print("Prediction WITHOUT normalization:", pred_wo)
print("Prediction WITH normalization:", pred_w)
```

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Explanation

- create_model()
- model.compile()
- model.fit()
- model.evaluate()
- model.predict()

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

In create_model() Function

- model = tf.keras.Sequential([

  tf.keras.layers.Dense(8, activation='relu', input_shape=(2,)),
  tf.keras.layers.Dense(4, activation='relu'),
  tf.keras.layers.Dense(1, activation='sigmoid')

  ])

- **tf.keras.layers.Dense(8, activation='relu', input_shape=(2,)),**
  - This is the first hidden layer
  - Dense(8): Fully connected layer, 8 neurons, Each neuron connects to all inputs
  - input_shape=(2,): Model expects 2 input features (Example: study hours, attendance), Required only for the first layer
  - activation='relu’: Applies ReLU activation

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

In create_model() Function

- **tf.keras.layers.Dense(4, activation='relu’),**
  - This is the second hidden layer
  - Dense(4): 4 neurons, Learns higher-level features from previous layer
  - activation='relu’: Maintains non-linearity, Enables deeper learning
- **tf.keras.layers.Dense(1, activation='sigmoid')**
  - Dense(1): Single neuron, Produces one output value
  - activation='sigmoid’
    - Why sigmoid?
      - Output range: 0 to 1
      - Interpretable as probability
      - Ideal for binary classification

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

model.compile()

- **model.compile(optimizer='adam’, loss='binary_crossentropy’, metrics=['accuracy’])**
  - This step configures the learning process of the neural network.
  - optimizer='adam’ : Optimizer updates weights to minimize loss
  - loss='binary_crossentropy’ : Measures how wrong the prediction is
  - metrics=['accuracy’] : Tells TensorFlow what to display during training

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

model.fit()

- **model.fit(X, y, epochs=50, verbose=0)**
  - This function trains the neural network.
  - X : Input features (training data)
  - y: True output labels
  - epochs=50 : One epoch = one complete pass through the dataset, 50 epochs = model learns 50 times
  - verbose=0: Controls output display: 0 → silent, 1 → progress bar, 2 → one line per epoch

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

model.evaluate()

- **loss, accuracy = model.evaluate(X, y, verbose=0)**
  - Evaluates the trained model on **test or training data**.
  - **verbose=0:** Suppresses console output

model.predict()

- **prediction = model.predict(test_input)**
  - Generates **predictions for new/unseen data**.
  - Returns a **probability value** (because sigmoid is used)
  - Prediction = 0.82, Interpretation: ≥ 0.5 → Pass, < 0.5 → Fail

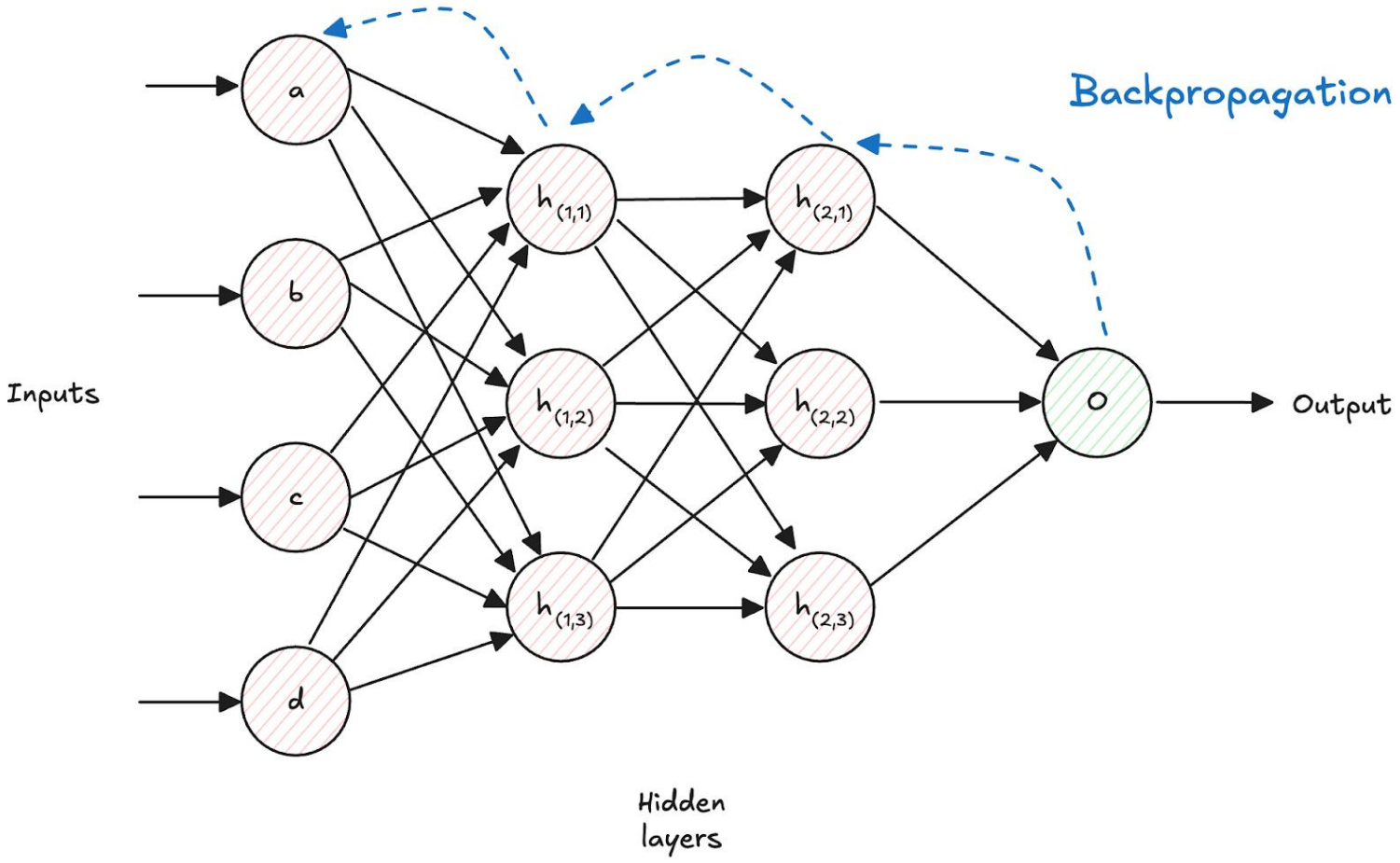
This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

create_model() → compile() → fit() → evaluate() → predict()

Function	Role
create_model	Builds architecture
compile	Configures learning
fit	Trains model
evaluate	Tests performance
predict	Makes predictions

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Back propagation in neural networks



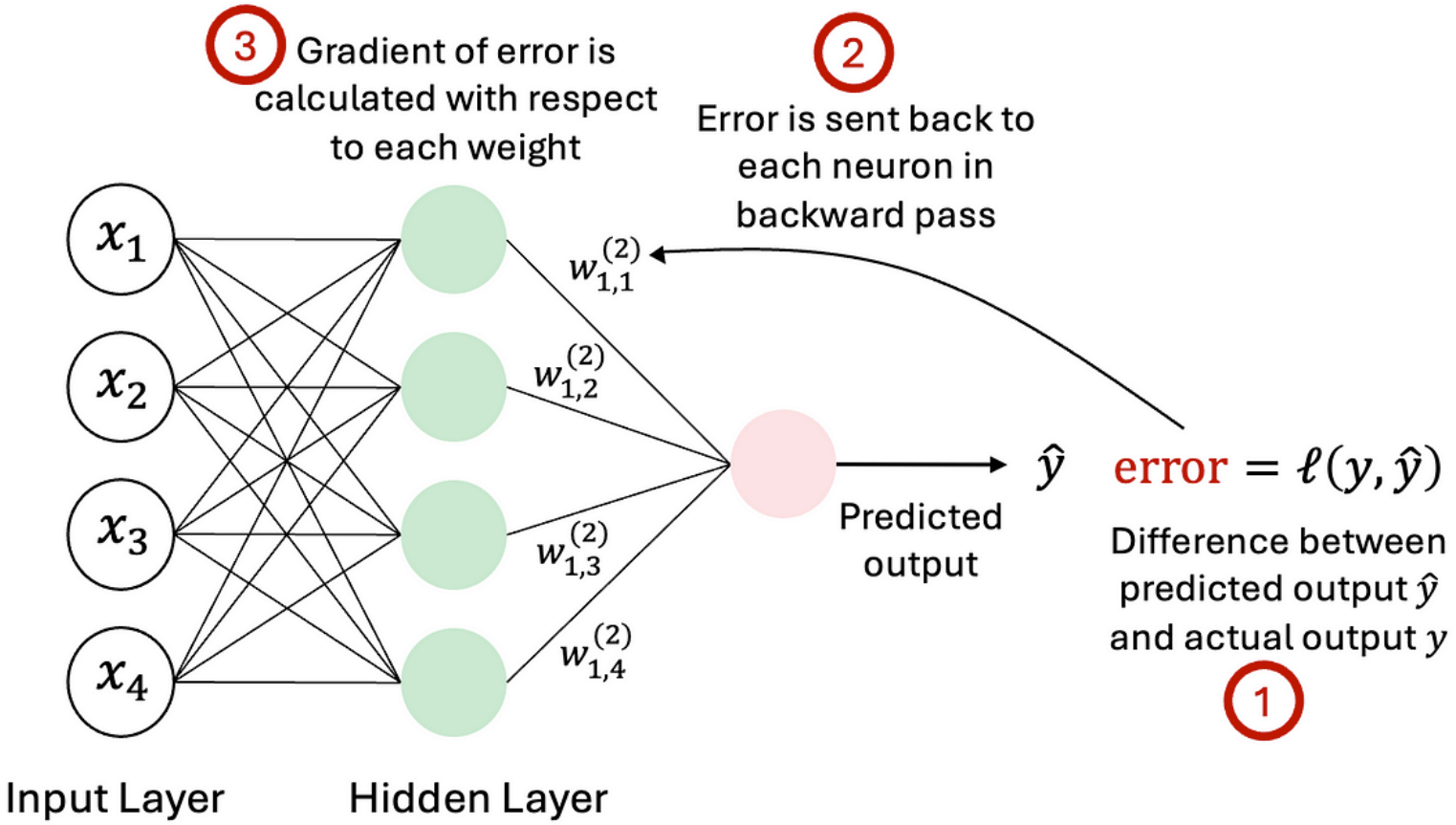
This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

BPN(Back Propagation Network) 1986

- Definition:
  - Network associated with back propagation learning algorithm is called BPN
- Multi layer feed forward Neural Network
- Uses Gradient-descent based delta-learning rule
- Errors are propagated back to the hidden unit
- Minimize the total squared error of the output
- Aim:
  - To train the net to achieve a balance between the ability to respond correctly to the input patterns and the ability to provide good responses to the input that are similar

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Steps Back propagation in neural networks



This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

BPN(Back Propagation Network) 1986

- Each neuron computes:

$$z = \sum w_i x_i + b, a = f(z)$$

- where $f(\cdot)$is an activation function (ReLU, sigmoid, tanh, etc.).

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

How Backpropagation Works

- **Step 1: Forward Pass**
  - Inputs → hidden layers → output
  - Compute predicted output $\hat{y}$
- **Step 2: Loss Computation**
  - Measure error using a **loss function**
    - Regression: MSE
    - Binary classification: Binary Cross-Entropy
    - Multi-class: Categorical Cross-Entropy
- **Step 3: Backward Pass**
  - Apply the **chain rule** to compute gradients of the loss w.r.t. each weight:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w}$$

- **Step 4: Weight Update (Gradient Descent)**

$$w_{new} = w_{old} - \eta \frac{\partial L}{\partial w}$$

where η is the **learning rate**.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Simple Mathematical View (Single Neuron)

- **Error**:

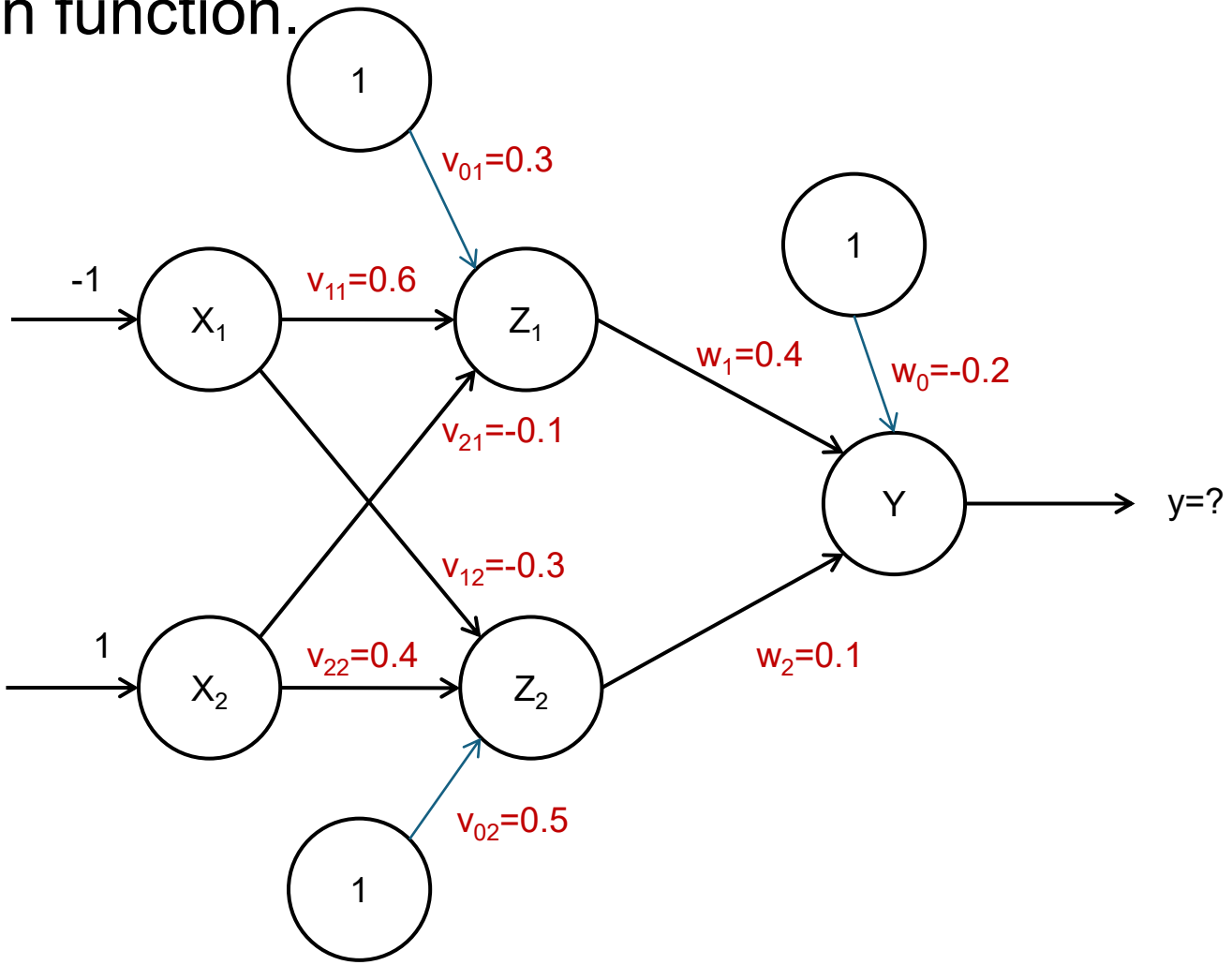
$$E = \frac{1}{2}\left(y - \hat{y}\right)^2$$

- **Gradient**:

$$\Delta w = -\eta(y - \hat{y})f'(z)x$$

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Find the new weights using back-propagation network for the network shown in Figure. The network is presented with the input patter [-1,1] and the target output is +1. Use a learning rate of α = 0.25 and bipolar sigmoidal activation function.



This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Solution:

The initial weights are,

v_{11}=0.6, v_{21}=-0.1, v_{01}=0.3 ; v_{12}=-0.3, v_{22}=0.4, v_{02}=0.5; w_1=0.4, w_2=0.1, w_0=-0.2

Activation function, binary sigmoidal function,

$$f(x) = 1- e^{-x} / 1+e^{-x}$$

x_1 = -1, x_2=1 and t=1

Step-1: Calculate the net input

For z_1 layer, $z_{in1} = v_{01}+ x_1.v_{11} + x_2.v_{21} = 0.3 + (-1) \times 0.6 + 1 \times -0.1 = -0.4$

For z_2 layer, $z_{in2} = v_{02}+ x_1.v_{12} + x_2.v_{22} = 0.5 + (-1) \times -0.3 + 1 \times -0.1 = 1.2$

Step-2: Calculate output by applying activation function

$z_1= f(z_{in1}) = 1- e^{-z_{in1}} / 1+e^{-z_{in1}} = 1 – e^{-(-0.4)} / 1+e^{-(-0.4)} = -0.1974$

$z_2= f(z_{in2}) = 1- e^{-z_{in2}} / 1+e^{-z_{in2}} = 1 – e^{-1.2} / 1+e^{-1.2} = 0.537$

Step-3: Calculate the netinput at output layer

$y_{in} = w_0+ z_1.w_1 + z_2.w_2 = -0.2 + (-0.1974) \times 0.4 + 0.537 \times 0.1 = -0.22526$

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Step-4: Calculate output of output layer

Apply Activation function, binary sigmoidal function,

$y = f(y_{in}) = 1 - e^{-y_{in}} / 1 + e^{-y_{in}} = 1 - e^{-(-0.22526)} / 1 + e^{-(-0.22526)} = -0.1122$

Step-5: Compute the error portion δ_k

$\delta_k = (t_k - y_k) f^{I}(y_{ink})$

$f^{I}(y_{in}) = 0.5[1 + f(y_{in})][1 - f(y_{in})] = 0.5[1-0.1122][1+0.1122] = 0.4937$

$\therefore \delta_3 = (1+0.1122)(0.4937) = 0.5491$

Step-6: So, update weight between hidden and output layer

$w_1 = w_1(old) + \alpha\delta_3 z_1 = 0.4 + 0.25 \times 0.5491 \times -0.1974$ **= 0.3729**

$w_2 = w_2(old) + \alpha\delta_3 z_2 = 0.1 + 0.25 \times 0.5491 \times 0.537$ **= 0.1737**

$w_0 = w_0(old) + \alpha\delta_3 = -0.2 + 0.25 \times 0.5491$ **= -0.0627**

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Step-7: Computer error portion δ_j (j=1 to 2)between input and hidden layer

$\delta_j = \delta_{inj} f^{I}(z_{inj})$

$\delta_{inj} = \delta_3 w_{j1} \rightarrow \quad \delta_{in1} = \delta_3 w_1 = 0.5491 \times 0.4 = 0.21964$

$\delta_{in2} = \delta_3 w_2 = 0.5491 \times 0.1 = 0.05491$

Error, $\delta_1 = \delta_{in1} f^{I}(z_{in1}) = 0.21964 \times 0.5x(1+0.1974)(1-0.1974)$ = **0.1056**

Error, $\delta_2 = \delta_{in2} f^{I}(z_{in2}) = 0.05491 \times 0.5x(1-0.537)(1+0.537)$ = **0.0195**

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Step-8: Now update weight between hidden and input layers

$v_{11} = v_{11}(old) + \alpha\delta_1 x_1 = 0.6 + 0.25 \times 0.1056 \times -1$ **= 0.5736**

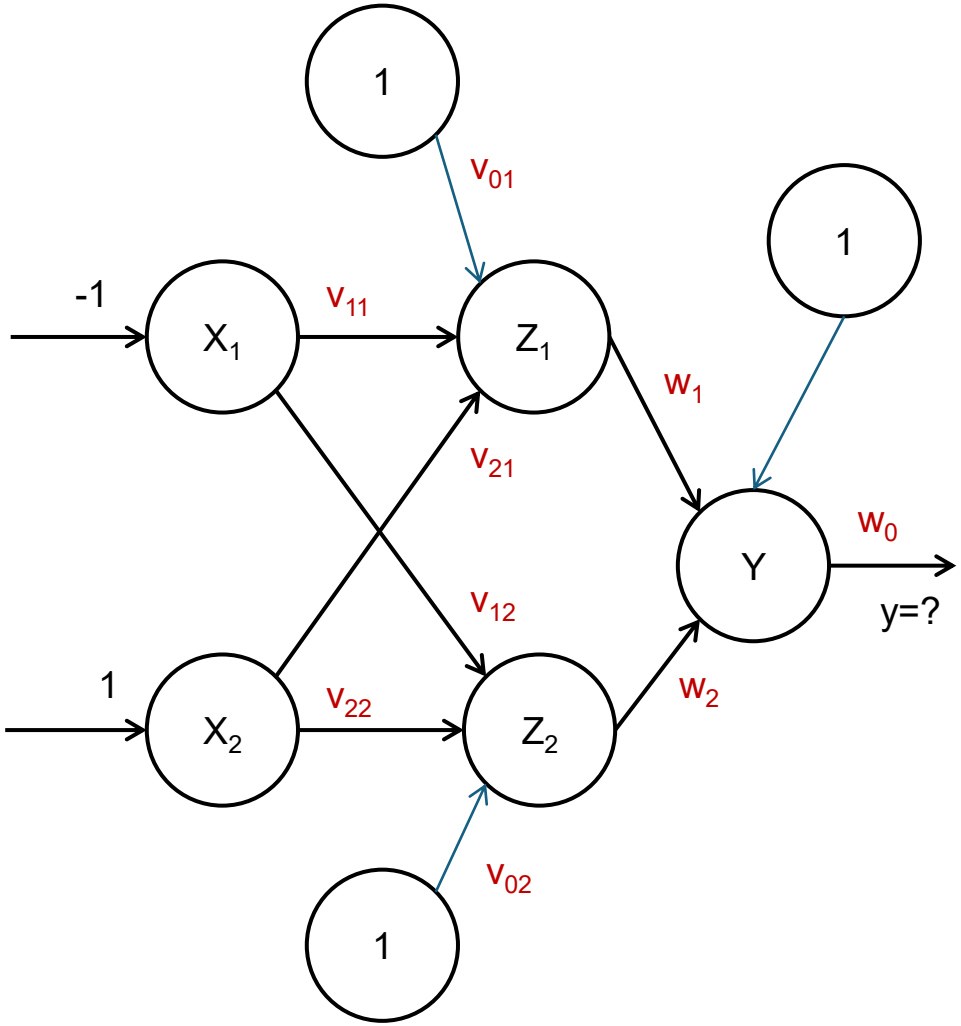
$v_{21} = v_{21}(old) + \alpha\delta_1 x_2 = -0.1 + 0.25 \times 0.1056 \times 1$ **= -0.0736**

$v_{01} = v_{01}(old) + \alpha\delta_1 = 0.3 + 0.25 \times 0.1056$ **= 0.3264**

$v_{12} = v_{12}(old) + \alpha\delta_2 x_1 = -0.3 + 0.25 \times 0.0195 \times -1$ **= -0.3049**

$v_{22} = v_{22}(old) + \alpha\delta_2 x_2 = 0.4 + 0.25 \times 0.0195 \times 1$ **= 0.4049**

$v_{02} = v_{02}(old) + \alpha\delta_2 = 0.5 + 0.25 \times 0.0195$ **= 0.5049**



Final weights of the network are as follows:

v_{11} **= 0.5736,** v_{21} **= -0.0736,** v_{01} **= 0.3264**

v_{12} **= -0.3049,** v_{22} **= 0.4049,** v_{02} **= 0.5049**

w_1 **= 0.3729,** w_2 **= 0.1737,** w_0 **= -0.0627**

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Chain rule

- In a neuron

$$z = wx + b$$
$$a = f(z)$$
$$L = Loss(a)$$

- Here
  - Loss Ldepends on activation a
  - $a$depends on z
  - $z$depends on weight w

- So

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w}$$

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Loss functions

- Binary cross entropy loss
- Categorical cross entropy loss

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Loss functions

- **Binary Cross-Entropy**
  - “Binary cross-entropy loss measures the error between true binary labels and predicted probabilities produced by a sigmoid activation function.”
- **Categorical Cross-Entropy**
  - “Categorical cross-entropy loss evaluates the dissimilarity between true one-hot encoded class labels and predicted probability distributions generated by softmax.”

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Binary cross entropy loss

- **Binary Cross-Entropy loss** measures how well a model predicts a **binary outcome (0 or 1)** by comparing the predicted probability with the true label.
- It penalizes confident but wrong predictions **very heavily**.
- **Equation**

  For a single data point:

$$L = -[y\log(p) + (1-y)\log(1-p)]$$

  Where:

  $y \in \{0, 1\}$ →true label

  p→ predicted probability (from **sigmoid**)

- **Intuition**
  - If the true label is **1**, the loss depends on $\log(p)$
  - If the true label is **0**, the loss depends on $\log(1-p)$

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Binary cross entropy loss

- **Case 1: Correct prediction**
  - True label $y = 1$
  - Predicted probability $p = 0.9$

$$L = -\log(0.9) \approx 0.105$$

  - Result: Small loss

- **Case 2: Wrong prediction**
  - True label $y = 1$
  - Predicted probability $p = 0.1$

$$L = -\log(0.1) \approx 2.303$$

  - Large loss (strong penalty)

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Categorical cross entropy loss

- Categorical Cross-Entropy loss measures the difference between the true class distribution and the predicted probability distribution in multiclass classification.
- Equation

$$L = -\sum_{i=1}^{C} y_i \log(p_i)$$

Where

C → number of classes

y_i →true label (one-hot encoded)

p_i →predicted probability (from softmax)

- **Intuition**
  - Only the probability of the **correct class** matters
  - If the model assigns low probability to the correct class → **high loss**

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Categorical cross entropy loss

- Problem: Digit classification (0, 1, 2)
  - True label: Class 2
  - One-hot encoding:

$$y = [0,0,1]$$

  - Predicted probabilities (Softmax):

$$p = [0.2, 0.3, 0.5]$$

  - Loss:

$$L = -\log(0.5) \approx 0.693$$

  - **Moderate loss**
- **Wrong prediction example**

$$p = [0.6,0.3,0.1]$$

$$L = -\log(0.1) \approx 2.303$$

  - Large loss

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Binary vs Categorical Cross-Entropy

Aspect	Binary Cross-Entropy	Categorical Cross-Entropy
Problem type	Binary	Multiclass
Output neurons	1	≥2
Activation	Sigmoid	Softmax
Labels	0 / 1	One-hot vectors
Example	Spam / Not spam	Digit (0–9)

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Lab 1: exploring the environment of tensor flow

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Lab 2: Building Programs to Perform Basic Operations in Tensors, regression

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.